Plain versus not plain

A verification walkthrough of the binary pricing rewrite

Consumer objective, firm information, optimization, code syntax, and links to the theory

> The earlier results in `new_pricing_res/` came from the full-basket model. They are **not results for the binary specification requested here**. The corrected binary solver is implemented and mathematically tested; its empirical results require a new server run.

This document explains what the corrected code does, including the assumptions needed to reuse the existing estimates. It does not claim that a binary demand model has been estimated.

Read Sections 1–4 first to check the model itself. Sections 5–13 connect every calculation to Python syntax and equations. The appendices contain complete active pricing sources and the reused dependencies, with line numbers and frozen source hashes.

All material here is source code, estimates, aggregate results from the earlier run, or explicitly labeled mathematical examples. No household records or purchase histories were read or generated locally.

Verification version: November 11, 2026.

Presentation flavor: plain. The same code can run berry and other yogurt as the inside alternative.

Contents

1 The model you asked for, in one page **4**

2 The outside option: what is combined, and what is fixed **4**

2.1 How switching is kept separate 5

2.2 The transport assumption you need to inspect 5

3 Direct comparison with the theoretical supply exercise **6**

3.1 How habit changes the optimum 6

4 What is reused from estimation, and what is not reestimated **7**

4.1 Saved parameters 7

4.2 Posterior means treated as known 8

4.3 What happens to household-specific novelty estimates? 8

4.4 What the likelihood check establishes 8

5 Starting environments: the role of actual observed histories **9**

6 How the novelty distribution enters the calculation **10**

6.1 Mean and standard deviation generate a distribution, not one average type 10

6.2 Population crossing is an explicit experiment 10

6.3 Low/middle/high groups and prior sensitivity 10

7 Step one of backward induction: the two informed terminal prices **11**

8 Step two: the informed first price **11**

9 Limited information: two common terminal prices **12**

9.1 Numerical details and the rounding failure 12

10 Limited information: the first price and the meaning of fixed beliefs **13**

10.1 The distinction most important to verify 13

11 Policy evaluation: consumers keep their type **14**

11.1 Why the belief objective and evaluated revenue differ 14

12 A complete numerical example you can follow by hand **14**

13 The server loop, line by line in economic terms **15**

13.1 Mapping diagnostics: an important new export 16

14 Python syntax: how to read the array calculations **16**
14.1 The exact two-state return value . . . 17

15 Aggregation: from solved prices to exported means **17**
15.1 Expected terminal prices . . . 17
15.2 Price changes and ratios . . . 18
15.3 Revenue conditional on a purchase history . . . 18

16 How the figures mirror supply, and how to interpret them **18**

17 What the earlier results were, and why they need rerunning **19**

18 What has been checked, and what remains a choice **19**

19 Verification checklist before interpreting the rerun **20**

20 How to run all three flavors and update plain in the presentation **21**

21 Code reading map and frozen sources **21**

A Complete binary solver **23**

B Reused mixture-price optimizer **25**
B.1 `_terminal_common` . . . 25

C Complete server orchestration and aggregation **26**

D History rules: only saved cumulative history is accepted by the new binary runner **30**

E Reused posterior and starting-context functions **31**
E.1 `logsumexp` . . . 31
E.2 `persistent_draws` . . . 31
E.3 `posterior` . . . 31
E.4 `forecast_context` . . . 32

F Saved-data reconstruction dependencies **33**
F.1 `normalized_upc_expr` . . . 33
F.2 `read_flavors` . . . 33
F.3 `prepare_data` . . . 34
F.4 `check_dma` . . . 36
F.5 `switch_away_event` . . . 37
F.6 `add_stocks` . . . 37

G Flavor ordering and classification definitions **39**

H Complete aggregate figure generation **40**

I Complete aggregate import into the presentation **43**

J Complete server submission launcher **45**

K Complete server data policy **46**

L Complete binary theoretical information exercise **47**

M Theoretical numerical helper functions **50**

M.1 `expit` 50

M.2 `golden_maximum` 50

M.3 `static_price` 50

N Complete binary regression and mathematical tests **51**

O Previous full-basket solver: the source of the earlier results **53**

1 The model you asked for, in one page

Let j denote the selected inside flavor. For the presentation, j is plain. There are exactly two alternatives and two possible first-period outcomes:

$$x_t \in \{j, 0\}, \qquad 0 = \text{not plain: berry, other yogurt, or nonpurchase.}$$

The initial history h_0 and the new history h_1 each record only inside or outside. The model never sets a different second price after berry versus other yogurt versus nonpurchase. It chooses $p_2(j)$ or $p_2(0)$.

A consumer's current relative utility is

$$v_{ir,t} = b_i + \gamma_r q(h_t) + \phi_i H_{it} - a_i p_{it}, \qquad q(j) = -1, \quad q(0) = +1, \quad a_i > 0. \tag{1}$$

The current purchase probability is $s_{ir,t} = \text{logit}^{-1}(v_{ir,t})$. Consumers make the current choice using current utility; they are not solving their own forward-looking consumption problem. The firm is forward-looking over two periods.

Own-flavor habit updates by

$$H_{i,t+1} = H_{it} + \mathbf{1}\{x_t = j\}. \tag{2}$$

For a given starting H_{i1}, there are just two successor states:

$$(h_1, H_{i2}) = (j, H_{i1} + 1) \quad \text{or} \quad (0, H_{i1}).$$

Habit adds a continuous state to the binary model, but each starting environment still has exactly two next-period branches.

	Perfect information	Limited information
Initial history, price sensitivity, habit	Known	Known
Novelty γ	Known value	Known distribution
First price	$p_{1r}(h_0)$	$p_1(h_0)$
Second price	$p_{2r}(h_1)$	$p_2(h_1)$
Observe first purchase before second price	Yes	Yes
Update beliefs about novelty	Not needed	No: fixed weights

Prices are continuous. The first price may be above or below the observed price and can be negative. The terminal optimum is nonnegative at zero marginal cost. There is no restriction leaving a middle type at its observed price.

Decision to verify

Is this the intended consumer objective? In particular: (i) one binary purchase opportunity in each period; (ii) the symmetric theory switching effect $+\gamma$ after outside and $-\gamma$ after inside; (iii) own habit increases only after an inside choice; (iv) only novelty is hidden.

2 The outside option: what is combined, and what is fixed

There is one consumer-specific novelty preference, not separate novelty parameters for berry, plain, and other yogurt. There is therefore no averaging of different flavor-specific γ coefficients.

The code first combines the *baseline* utilities of the available non-focal alternatives. Write $\bar{\beta}_k$ for the fitted flavor intercept, p^0_{ik} for its observed price, S^0_{ik} for its starting stock, and c_{ik} for an optional fixed utility control. Then

$$B_{ik} = \bar{\beta}_k - a_i p^0_{ik} + \phi_i S^0_{ik} + c_{ik}, \qquad k \neq j, \tag{3}$$

$$C_i = \log\left(1 + \sum_{k \neq j,\, k \text{ available}} e^{B_{ik}}\right), \tag{4}$$

$$b_i = \bar{\beta}_j + c_{ij} - C_i. \tag{5}$$

The 1 in (4) is e^0, the nonpurchase alternative. Unavailable flavors are omitted. For plain, the sum includes berry and other yogurt when their prices are available.

This is a log-sum-exp, not an arithmetic mean of utility, price, or novelty. For any specified utilities,

$$\frac{e^{V_j}}{e^{V_j} + \sum_{k \neq j} e^{V_k}} = \text{logit}^{-1}\left(V_j - \log \sum_{k \neq j} e^{V_k}\right).$$

That algebra explains how several alternatives can form one outside index.

2.1 How switching is kept separate

The two binary deterministic utilities can be written

$$V_{j,t} = \bar{\beta}_j + c_{ij} - a_i p_t + \phi_i H_{it} + \gamma_r \mathbf{1}\{h_t = 0\}, \tag{6}$$

$$V_{0,t} = C_i + \gamma_r \mathbf{1}\{h_t = j\}. \tag{7}$$

Subtracting $V_{0,t}$ gives (1). Every non-focal choice receives the same binary switching treatment. The code excludes novelty from C_i because it is represented explicitly by these switching terms.

```
rivals = np.flatnonzero(available & (np.arange(j) != focal))
rival_base = beta[rivals]-alpha*prices[rivals] \
             + habit*stock[rivals]+controls[rivals]
outside_index = float(logsumexp(np.r_[rival_base, 0.]))
intercept = float(beta[focal]+controls[focal]-outside_index)
```

Here `np.r_[rival_base, 0.]` appends nonpurchase utility zero to the vector of rival baseline utilities. `logsumexp` computes their inclusive value stably, without directly exponentiating large numbers.

2.2 The transport assumption you need to inspect

The baseline index C_i, rival prices, and rival habit stocks are held fixed during the two pricing periods. Only the inside habit stock and the binary last-choice indicator evolve. This makes the future history genuinely binary.

> This is an explicit binary counterfactual using estimates from the original multinomial demand model. It is not an exact dynamic aggregation of that fitted model. In the original model, novelty and habit of every flavor can change after a basket. Here, outside baseline utility is frozen and novelty follows the theory's symmetric binary rule.

The original fitted own-flavor novelty index is a past purchase-share measure. The rewrite uses the same numerical coefficient in $\gamma q(h)$, with $q \in \{-1, +1\}$. The difference between the two binary histories is

therefore 2γ. No automatic factor-of-two rescaling or binary reestimation is performed. That parameter transfer is a modeling assumption, not something established by the saved likelihood check.

Decision to verify

Are you comfortable fixing the outside baseline for two periods and transporting the estimated novelty coefficient directly into the theory's binary switching rule? If the target is exact dynamic aggregation of the original fitted multinomial model, this binary specification does not deliver that different target.

3 Direct comparison with the theoretical supply exercise

The theoretical relative utility in the existing supply code is

$$v_r^T(p, h) = \beta + \gamma_r q(h) - ap, \qquad h \in \{j, 0\}.$$

Its numerical script uses `ALPHA=-a` and writes `BETA + ALPHA*p`. The rewritten empirical code uses positive `alpha=a_i` and subtracts `alpha*p`. These are equivalent sign conventions.

Object	Theoretical example	Binary empirical rewrite
Choice	Inside or outside	Selected flavor or all other choices
Choice opportunities	One per period	One per period
History after first choice	j or 0	j or 0
Baseline intercept	Common illustrative β	Known $b_i = \bar{\beta}_j + c_{ij} - C_i$
Price sensitivity	Common illustrative a	Known estimated a_i
Novelty	Illustrative discrete support	Nine nodes approximating fitted normal population
Habit	Absent	Known ϕ_i, evolving inside stock H_i
Outside attractiveness	Normalized baseline zero	Fixed inclusive value, subtracted into b_i
Firm horizon	Two periods	Two periods
Revenue discount	$\delta = 1$ in the displayed example	$\delta = 1$ by default
Marginal cost	Zero	Zero
Limited information	Fixed type weights in each state	Same fixed-weight rule
Policy evaluation	Same type through both periods	Same type through both periods

The binary rewrite reduces exactly to the theoretical objective when $\phi_i = 0$, $b_i = \beta$, and $a_i = a$. The regression test checks both starting histories, every informed type, the common first price, both common terminal prices, and the limited firm's subjective objective against the separate theoretical implementation.

3.1 How habit changes the optimum

The two period-two relative utility intercepts are

$$z_{ir,j} = b_i - \gamma_r + \phi_i(H_{i1} + 1), \qquad z_{ir,0} = b_i + \gamma_r + \phi_i H_{i1}. \tag{8}$$

The initial intercept is

$$z_{ir,1} = b_i + \gamma_r q(h_0) + \phi_i H_{i1}. \tag{9}$$

Thus habit is additive in current utility, but it changes the continuation value after an inside choice. It cannot be removed from the dynamic calculation while claiming to preserve the habit-inclusive objective. Negative γ rewards repeating the most recent binary choice; ϕH is a distinct accumulated-exposure effect. ϕ can itself be negative.

```
history_sign = -1. if initial_inside else 1.
intercept1 = intercept+gamma*history_sign+habit*own_stock
intercept2 = np.column_stack([
    intercept-gamma+habit*(own_stock+1.),
    intercept+gamma+habit*own_stock])
```

The two columns are ordered *after inside, after outside*. They are not berry and other yogurt. `np.column_stack` makes a $G \times 2$ matrix when there are G novelty nodes.

4 What is reused from estimation, and what is not reestimated

4.1 Saved parameters

The aggregate parameter file below has the same SHA-256 as the parameter file recorded in the earlier completed server run. It is safe to inspect these estimates locally. The new binary run has not yet produced results.

Parameter	Saved estimate
inside_constant	1.217332040
beta_berry	-0.624098524
beta_plain	-0.271941079
mu_log_minus_alpha	-0.463007469
mean_novelty	-0.773240784
mean_own_stock	0.036901728
sd_log_minus_alpha	0.837837833
sd_novelty	0.800316442
sd_own_stock	0.078284064

The pricing vector of fitted flavor intercepts is

$$\bar{\beta} = (\beta_{inside},\ \beta_{inside} + \beta_{berry},\ \beta_{inside} + \beta_{plain}),$$

ordered *other, berry, plain*. For plain the fitted baseline intercept before outside normalization is approximately 0.945391; it is not the older binary calibration intercept 10.502.

The saved random-coefficient mapping is

$$a_d = \exp(\mu_a + \sigma_a z_{ad}) > 0, \tag{10}$$

$$\gamma_d = \mu_\gamma + \sigma_\gamma z_{\gamma d}, \tag{11}$$

$$\phi_d = \mu_\phi + \sigma_\phi z_{\phi d}. \tag{12}$$

The saved population distributions are independent in these latent draws. Conditioning on observed choices can make posterior coefficient beliefs correlated.

4.2 Posterior means treated as known

The server reconstructs the original estimation input and computes, for draw d,

$$\ell_{id} = \sum_t \sum_k y_{ikt} \log P_{ikt}(a_d, \gamma_d, \phi_d).$$

Constants not depending on the draw are unnecessary for posterior weights. With equally weighted integration draws,

$$\rho_{id} = \frac{e^{\ell_{id}}}{\sum_{d'} e^{\ell_{id'}}}, \tag{13}$$

$$\widehat{a}_i = \sum_d \rho_{id} a_d, \qquad \widehat{\phi}_i = \sum_d \rho_{id} \phi_d. \tag{14}$$

These two posterior means enter *both* pricing regimes as if known. They are not actual directly observed tastes and their estimation uncertainty is not integrated in pricing.

```
weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
table["alpha_mean"] = weights @ alpha  # constructed in the table
# The implementation uses the same weighted sum for habit:
table["habit_mean"] = weights @ habit
```

The first assignment to `alpha_mean` is embedded in a `DataFrame` constructor in the actual source; the displayed line spells out its operation. The complete source is in the appendix.

Decision to verify

The firm is assumed to know these nuisance tastes at the pricing date even though they are estimated using the full observed sample, including later choices. This is a retrospective information experiment, not a claim that a new customer's tastes could be known from the first purchase alone.

4.3 What happens to household-specific novelty estimates?

The reused posterior function also computes each household's posterior mean novelty and assigns household quartile groups for the older exercise. **The new information exercise does not use those household-specific novelty estimates as its true types.** It crosses a population novelty distribution with every eligible known-price/habit/history environment. Its low/middle/high display groups are groups of population integration nodes, not those old household quartile assignments.

4.4 What the likelihood check establishes

The runner requires the reconstructed old-model likelihood to match the saved value. In the earlier completed run both values were 1,627,493.8833353105. This validates reconstruction of the *saved multinomial fit*. It does not validate the new binary parameter mapping or show that binary demand has been estimated.

The workflow does not rerun the optimizer that estimated the original demand parameters. Source for the needed reconstruction and posterior calculations is included; unrelated old estimation optimization code is not an active part of this pricing job.

5 Starting environments: the role of actual observed histories

An environment consists of a known starting menu, observed prices, saved stocks, and the nuisance posterior means in (14). Only this reconstruction is performed on the research server. No such individual environments are exported locally.

1. Reconstruct the saved estimation sample using its original settings.
2. Construct cumulative history before exclusions of unusable weeks.
3. Keep households with the required usable weeks and verify the likelihood.
4. Take each household's earliest retained estimation row.
5. Carry its menu, rival prices, and fixed utility controls forward for two pricing periods.
6. Retain an environment for a focal flavor only when that flavor has a finite positive price in the starting menu.

The initial binary history is

$$h_{i0} = j \quad \text{if any focal-flavor units occur in that starting row;} \qquad h_{i0} = 0 \text{ otherwise.}$$

A mixed initial basket containing plain is therefore initially inside plain. This initialization rule does not require the initial observed week to have been a single binary purchase. The two *counterfactual pricing decisions* that follow it are binary.

If the saved pre-purchase inside stock is S_{ij,t_0} and the initial observed share is w_{ij,t_0}, the stock entering the first priced period is

$$H_{i1} = S_{ij,t_0} + w_{ij,t_0}.$$

It need not be zero or an integer. Earlier history can remain in S because stocks are built before unusable-price rows are filtered. Future binary purchases add one or zero.

> The rewrite accepts the saved cumulative-history specification only. It does not silently apply the newer average-stock default to old estimates. The original estimation uses weekly counts with capacity seven for the nonpurchase construction; the binary counterfactual uses one theoretical choice opportunity per pricing period. The unit and transition change are explicitly recorded in the new run summary.

The previous full-basket plain run had 1,839 eligible environments: 1,324 initially inside and 515 initially outside. These are counts of underlying eligible household setups, not 1,839 aggregate preference groups. The local exported files are aggregates over these setups. Coverage for the corrected all-flavor run must be read from its new metadata; it has not been observed yet.

An inside/outside comparison of averaged outcomes uses different observed environments. It is not automatically a ceteris paribus change of h_0 holding menus, prices, and nuisance tastes fixed, as in a single illustrative theoretical calibration.

6 How the novelty distribution enters the calculation

6.1 Mean and standard deviation generate a distribution, not one average type

For $Z \sim N(0,1)$ and $\gamma = \mu_\gamma + \sigma_\gamma Z$, the code approximates

$$\mathbb{E}[f(\gamma)] \simeq \sum_{r=1}^{G} w_r f(\gamma_r), \qquad \gamma_r = \mu_\gamma + \sqrt{2}\sigma_\gamma z_r, \quad w_r = \frac{w_r^{GH}}{\sqrt{\pi}}. \tag{15}$$

Here z_r, w_r^{GH} are Gauss–Hermite nodes and weights for integrating against e^{-z^2}. The default is $G = 9$.

```
nodes, weights = hermgauss(args.nodes)
gamma = parameters['mean_novelty'] \
        + np.sqrt(2)*parameters['sd_novelty']*nodes
weights = weights/np.sqrt(np.pi)
```

This is deterministic numerical integration, not nine sampled households or nine realized shocks. A type γ_r remains fixed through both periods.

Node	γ_r	Weight	Display group
1	-4.384865	0.00002235	low
2	-3.338598	0.00278914	low
3	-2.435376	0.04991641	low
4	-1.592169	0.24409750	low
5	-0.773241	0.40634921	middle
6	0.045688	0.24409750	high
7	0.888895	0.04991641	high
8	1.792117	0.00278914	high
9	2.838384	0.00002235	high

6.2 Population crossing is an explicit experiment

For every eligible environment i, the solver considers all nine types using the same weights w. The firm knows a_i, ϕ_i, H_i, C_i; only the population type γ_r is hidden in limited information. Thus the expectation is over a population distribution crossed with observed environments. It is not a forecast conditional on that household's posterior novelty distribution or on its past choices beyond the known environment.

Decision to verify

If the intended perfect-information case instead uses each household's estimated novelty posterior mean, or the limited-information prior should be conditional on initial history and known tastes, those are different experiments. The current crossing is the distribution-to-theory experiment implemented here.

6.3 Low/middle/high groups and prior sensitivity

The code partitions nodes using the continuous normal quartile cutoffs $\mu_\gamma \pm 0.67448975\sigma_\gamma$. Nine-node quadrature is not exact for threshold indicators, so the node masses are not exactly 25/50/25:

Group	Numerical mass	Mean γ
Low	0.296825	-1.750590
Middle	0.406349	-0.773241
High	0.296825	0.204108

For a sensitivity scenario favoring group g, target masses are $m_g = .5$ and .25 for the other groups. Within a group, relative node weights stay unchanged:

$$w'_r = w_r \frac{m_{g(r)}}{\sum_{s:g(s)=g(r)} w_s}.$$

Both policies are optimized and evaluated using the same scenario weights. Changing scenarios changes the hypothetical population as well as the firm's weights. These are sensitivity experiments, not estimates of different observed population shares. The fitted-distribution baseline is separate from an equal-three-type prior in the original illustrative theory.

7 Step one of backward induction: the two informed terminal prices

Fix an environment and drop i from notation. The terminal probability for type r after history $b \in \{j, 0\}$ is

$$s_{2r,b}(p) = \text{logit}^{-1}(z_{r,b} - ap), \qquad z_{r,b} \text{ given in (8)}.$$

At zero marginal cost, terminal revenue is $p\, s_{2r,b}(p)$. Differentiating,

$$\frac{d}{dp}[p\, s(p)] = s(p)\{1 - ap[1 - s(p)]\}.$$

The first-order condition is $ap = 1/[1 - s(p)]$. If $\omega(x)$ solves $\omega + \log \omega = x$, then

$$p^F_{2r,b} = \frac{1 + \omega(z_{r,b} - 1)}{a}, \qquad V^F_{2r,b} = p^F_{2r,b} s_{2r,b}(p^F_{2r,b}). \tag{16}$$

Equivalently, $\omega(x) = W(e^x)$, where W is Lambert W. `wrightomega` avoids constructing an overflow-prone exponential e^x.

```
p2_full = (1+wrightomega(intercept2-1))/alpha
r2_full = p2_full*expit(intercept2-alpha*p2_full)
```

Both arrays have G rows and exactly two columns. Every informed type gets a price after inside and a price after outside. There is no loop over yogurt baskets.

8 Step two: the informed first price

For type r, let $s_{1r}(p) = \text{logit}^{-1}(z_{r,1} - ap)$. Its two-period objective is

$$\Pi^F_r(p) = p s_{1r}(p) + \delta\{s_{1r}(p) V^F_{2r,j} + [1 - s_{1r}(p)] V^F_{2r,0}\}. \tag{17}$$

Define an effective first-period cost

$$c^{eff}_r = -\delta(V^F_{2r,j} - V^F_{2r,0}).$$

Then

$$\Pi^F_r(p) = \delta V^F_{2r,0} + s_{1r}(p)(p - c^{eff}_r),$$

and the solution is

$$p_{1r}^{F} = c_r^{eff} + \frac{1 + \omega(z_{r,1} - a c_r^{eff} - 1)}{a}. \tag{18}$$

This is the same continuation-value adjustment as in the supply first-order condition. A large benefit from an inside purchase lowers the effective cost and can induce an introductory subsidy. A disadvantage from that purchase raises it.

```
effective_cost = -delta*(r2_full[:, 0]-r2_full[:, 1])
p1_full = effective_cost + \
    (1+wrightomega(intercept1-alpha*effective_cost-1))/alpha
```

Revenue is optimized *separately for each novelty type*, then averaged. Averaging types before this maximization would solve the limited-information problem instead.

9 Limited information: two common terminal prices

The firm uses the same weights w_r after either outcome. For each terminal history b it chooses

$$p_{2,b}^{L} \in \arg\max_{p \geq 0} \; p \sum_r w_r s_{2r,b}(p), \qquad \bar{V}_{2,b}^{L} = \sum_r w_r p_{2,b}^{L} s_{2r,b}(p_{2,b}^{L}). \tag{19}$$

This is *one price across novelty types conditional on history*. It is not a single price across periods, and not one dollar price across all environments: known price sensitivity, habit, and outside attractiveness can differ across environments.

The common terminal-price first-order condition, in dimensionless price $t = ap$, is

$$D_b(t) = \sum_r w_r s_{r,b}(t)\{1 - t[1 - s_{r,b}(t)]\} = 0. \tag{20}$$

There is generally no single closed-form logit markup for the mixture. The optimizer brackets its candidates between the smallest and largest type-specific optimal prices. Below all individual optima each component derivative is nonnegative; above all of them it is nonpositive. A mixture can have multiple local maxima within that interval, so the code screens a grid and refines every detected positive-to-negative derivative crossing.

```
p2_limited, perceived2 = _terminal_common(
    intercept2, weights, alpha, 1, search_points)
```

The argument `1` is one binary purchase opportunity. The output prices and mixture values are length-two arrays, ordered after inside and after outside. The helper does not update w_r using purchases.

9.1 Numerical details and the rounding failure

The helper evaluates mixture shares on a dimensionless grid, bisects derivative crossings for 48 iterations, and compares objective values across candidate optima. If a refined stationary point and a grid point have objective values differing only within a scale-aware $64\epsilon_{machine}$ bound, it prefers the refined point. This prevents a rounded objective tie from retaining a grid point with a larger derivative residual.

The terminal derivative check remains at absolute tolerance 2×10^{-9}. The earlier screenshot's residual around 3.28×10^{-9} was reproduced using theoretical utility arrays and corrected through tie-breaking, not by removing that check. A grid search is numerical screening rather than a formal proof that every stationary point is detected for arbitrary inputs. The dense-search regression and wider-search options are checks of numerical accuracy, not statistical uncertainty intervals.

10 Limited information: the first price and the meaning of fixed beliefs

Write $\bar{s}_1(p) = \sum_r w_r s_{1r}(p)$. Under the specified fixed-belief rule, the first-period objective is

$$\Pi^L_{\text{belief}}(p) = p\bar{s}_1(p) + \delta\{\bar{s}_1(p)\bar{V}^L_{2,j} + [1 - \bar{s}_1(p)]\bar{V}^L_{2,0}\}. \tag{21}$$

This is the direct analogue of the theoretical script's `evaluate` function: use mixture demand in the first period and a fixed-belief mixture continuation value after each history.

Let $A = \delta(\bar{V}^L_{2,j} - \bar{V}^L_{2,0})$ and $p' = p + A$. Then

$$\Pi^L_{\text{belief}}(p) = \delta\bar{V}^L_{2,0} + p' \sum_r w_r \operatorname{logit}^{-1}\{(z_{r,1} + aA) - ap'\}.$$

The same mixture-price helper can optimize p', after which $p = p' - A$. Although $p' \geq 0$ at the terminal-style optimum, p can be negative.

```
adjustment = delta*(perceived2[0]-perceived2[1])
relative_price, relative_value = _terminal_common(
    intercept1[:, None]+alpha*adjustment,
    weights, alpha, 1, search_points)
p1_limited = np.full(len(gamma), relative_price[0]-adjustment)
limited_objective = relative_value[0]+delta*perceived2[1]
```

`intercept1[:, None]` converts a length-G vector into a $G \times 1$ matrix so that the helper solves one first-price problem. `np.full` repeats the common price across all possible types for evaluation. It does not give the firm knowledge of the true type.

10.1 The distinction most important to verify

The consumers keep their type, so a purchase generally changes the actual type composition of purchasers. Holding the firm's beliefs fixed is a restriction on its pricing behavior. Its subjective objective (21) need not equal the revenue actually produced by those prices.

For a Bayesian firm, weights after inside would be

$$w_{r|j}(p) = \frac{w_r s_{1r}(p)}{\sum_s w_s s_{1s}(p)},$$

and after outside they would be

$$w_{r|0}(p) = \frac{w_r[1 - s_{1r}(p)]}{\sum_s w_s[1 - s_{1s}(p)]}.$$

The current code *does not use these weights to choose prices.* It uses w_r in each terminal state and the matching subjective first-period objective.

Decision to verify

Does "limited information with fixed beliefs" mean exactly (19) and (21)? This is not the fully rational Bayesian optimum for an uninformed firm. Another possible specification would keep the fixed-belief terminal rule but choose the first price using correctly anticipated persistent-type revenue. That would also differ from the implemented first objective.

11 Policy evaluation: consumers keep their type

Once prices have been selected, the actual expected revenue for type r under policy $z \in \{F, L\}$ is

$$R_r^z = p_{1r}^z s_{1r}^z + \delta\{s_{1r}^z p_{2r,j}^z s_{2r,j}(p_{2r,j}^z) + (1 - s_{1r}^z) p_{2r,0}^z s_{2r,0}(p_{2r,0}^z)\}. \tag{22}$$

For common pricing, p_{1r}^L is identical across r, and the terminal prices are also identical across r within a history. Probabilities still differ across r.

$$R^z = \sum_r w_r R_r^z, \qquad \Delta R = R^F - R^L.$$

These expectations use the same r in both periods. They do not redraw novelty after the first purchase. The current choice shocks can be independent conditional on type while purchases are linked by the persistent type and history.

```
s1 = expit(intercept1-alpha*p1)
mass = np.column_stack([s1, 1-s1])
revenue1 = p1*s1
revenue2 = (mass*r2).sum(axis=1)
revenue = revenue1+delta*revenue2
```

`mass` is $G \times 2$, `r2` is $G \times 2$, and their elementwise product pairs each type's history probabilities with that *same type's* terminal revenue.

11.1 Why the belief objective and evaluated revenue differ

Let $V_{2r,b}^L = p_{2,b}^L s_{2r,b}(p_{2,b}^L)$. At the selected common policy,

$$R^L - \Pi_{\text{belief}}^L = \delta\{\text{Cov}_w(s_{1r}, V_{2r,j}^L) - \text{Cov}_w(s_{1r}, V_{2r,0}^L)\}.$$

This is an average-of-products versus product-of-averages difference. It is not a distinction between "total revenue" and "information value." Both price regimes have a revenue level, and their difference is the information gain under the stated restrictions.

The informed type can always choose the limited-information price path. Hence, with the same feasible prices and consumer model,

$$R_r^F \geq R_r^L \quad \text{for every type,} \qquad R^F \geq R^L.$$

This explains the dominance assertion. A positive gain by itself does not validate the demand specification, establish causal real-world gains, or demonstrate that the unknown-type policy is Bayesian optimal.

12 A complete numerical example you can follow by hand

This example is mathematical, not an empirical household. Take

$$b = 2, \quad a = 1, \quad \phi = .25, \quad H_1 = .5, \quad h_0 = 0, \quad \delta = 1, \qquad \Pr(\gamma = -1) = \Pr(\gamma = 1) = .5.$$

The outside baseline is normalized to zero in this example. The intercepts are

$$z_{r,1} = 2.125 + \gamma_r, \quad z_{r,j} = 2.375 - \gamma_r, \quad z_{r,0} = 2.125 + \gamma_r.$$

Work backwards: use (16) for each terminal state and type; compute $V_{2r,j}^F, V_{2r,0}^F$; form the effective cost and use (18). For limited information, maximize the two weighted terminal revenue functions, then the shifted mixture first-price problem.

	γ	p_1	$p_2(j)$	$p_2(0)$
Full information	-1	0.978971	2.791787	1.613529
Full information	1	3.025493	1.713114	2.633981
Limited information	hidden	2.277775	2.459846	2.339026

Full-information expected revenue is 2.254682. Limited-information realized expected revenue is 1.943313. The limited firm's subjective optimized value is 2.197045. The reported information gain is 0.311369.

Under the common first price, the low-type purchase probability is 0.239983 and the high-type purchase probability is 0.699985. The actual low-type share among purchasers is 0.255310, and among nonpurchasers it is 0.716975; the restricted firm nevertheless uses probability 0.5 in both states.

The limited firm can respond to *which binary history occurred* while still ignoring how that history changes the distribution of hidden novelty. That is exactly what "history-adaptive common prices with fixed beliefs" means in the code.

13 The server loop, line by line in economic terms

The full numbered source appears in the appendix. The following map covers every block of the orchestration file, including blocks with no economic equation.

Code operation	Meaning and verification purpose
Imports and path setup	Locate modules and the server policy. They do not estimate or simulate anything.
`@server_only`, `require_data_path`	Stop before empirical I/O on this computer; restrict server input/output paths.
Read `fit.json`, saved estimates	Load the old model's parameters and settings; reject missing, unconverged, or boundary fits.
Resolve saved history	Require cumulative stock for this binary rewrite. No use of an average-stock default.
`prepare_data`	Rebuild the saved estimation inputs on the server, including historical stocks and menus.
`posterior` and likelihood check	Recover known nuisance means and verify the old fitted likelihood.
`forecast_context`	Select the first retained observation and advance its stock once before pricing begins.
`hermgauss`	Build nine population novelty nodes and weights.
Build scenario weights	Keep the fitted distribution, or reweight one display group to one-half probability.
`flavor == 'all'`	Run every fitted flavor as j, including other yogurt; reconstruct data once.
Select eligible indices	Require the focal flavor in the starting price menu.
Initial state test	Any focal purchase in the starting row means $h_0 = j$; otherwise 0.
`solve_binary`	Compute all informed type prices, two common second prices, one common first price, and persistent-type revenues.
Compute mapping diagnostics	Compare binary and original multinomial probabilities at the observed price, using the same nuisance inputs.
Accumulate weighted metrics	Add sums indexed only by prior, initial state, regime, node and display group; never export individual policies.

Code operation	Meaning and verification purpose
`groupby(...).sum()`	Roll node sums into group and overall aggregates. No average is taken until dividing by the exported denominator.
`pivot` and information gain	Put informed and limited revenues side by side and subtract. Percent gains divide by limited revenue when positive.
Write summary, figures	Record model, source hashes, one-opportunity pricing unit, the outside mapping and aggregate outputs. Mark complete only after plotting succeeds.

13.1 Mapping diagnostics: an important new export

The new runner records aggregate columns `posted_probability_binary`, `posted_probability_multinomial`, and `posted_probability_gap`. For each environment and node these compare

$$s^B_{ir}(p^0_{ij}) = \text{logit}^{-1}(b_i + \gamma_r q(h_0) + \phi_i H_{i1} - a_i p^0_{ij})$$

with

$$s^M_{ir}(p^0_{ij}) = \frac{e^{\bar\beta_j - a_i p^0_{ij} + \gamma_r N^0_{ij} + \phi_i S^0_{ij} + c_{ij}}}{1 + \sum_{k \text{ available}} e^{\bar\beta_k - a_i p^0_{ik} + \gamma_r N^0_{ik} + \phi_i S^0_{ik} + c_{ik}}}.$$

Their difference is not assumed zero. A large aggregate gap would show that the binary mapping changes baseline choice behavior materially. It would be a reason to revisit the transport assumption; it is not repaired by a numerical optimizer tolerance.

14 Python syntax: how to read the array calculations

Indices are: environment i; novelty node $r = 1, \ldots, G$; flavor k; binary terminal state $b \in \{j, 0\}$. An environment loop runs on the server. The solver handles all novelty types for one environment simultaneously.

Syntax	Mathematical operation
`np.asarray(x, float)`	Convert input to a numeric array; does not average it.
`gamma.shape == (G,)`	A vector of possible persistent novelty coefficients.
`gamma[:, None]`	Add a dimension: G values become $G \times 1$ so one term can be used in each state column.
`intercept2.shape == (G,2)`	Every novelty type paired with exactly two histories.
`np.column_stack([a,b])`	Put vectors a and b in adjacent columns.
`p2_limited[None,:]`	One row with two common prices, broadcast across all novelty types.
`np.broadcast_to(...)`	Repeat values for arithmetic, not draw or observe extra types.
`A*B`	Elementwise multiplication, e.g. $P_{rb}V_{rb}$.
`weights @ values`	Dot product $\sum_r w_r \text{values}_r$.
`(mass*r2).sum(axis=1)`	Sum the two state contributions *within each type*.
`expit(x)`	$1/(1+e^{-x})$, implemented stably.
`logsumexp(x)`	$\log \sum_k e^{x_k}$, implemented stably.
`wrightomega(x)`	Positive real solution of $\omega + \log \omega = x$.
`np.where(condition,a,b)`	Choose a or b elementwise; used for availability and group labels.
`np.flatnonzero(mask)`	Indices of available non-focal products.

Syntax	Mathematical operation
`zip(gamma,weights,labels)`	Visit the aligned node, its probability, and its display group.
`dict(...)`	Named results, not a statistical distribution unless its values explicitly encode weights.
`sums.setdefault(key,...)`	Initialize an aggregate cell once, then add into it.
`table.groupby(keys).sum()`	Sum over omitted indices; means require a later denominator.
`frame.iloc[index]`	Select an in-memory row on the server. No rows are copied into this PDF.
`frame.loc[labels]`	Align source observations to the same household order as the nuisance means.
`np.testing.assert_allclose`	Numerical invariant check. Passing is not evidence of empirical identification.

14.1 The exact two-state return value

```
states = np.array([[1, 0], [0, 1]], dtype=int)
focal_index = 0
```

The first row means inside, the second outside. These rows are not weekly counts across three flavors. There are always two rows, even when the starting menu contains three or five fitted flavor categories. The runner uses `result['focal_index']` when forming branch aggregates, avoiding an accidental use of the original multinomial flavor-column index.

15 Aggregation: from solved prices to exported means

Let I_s denote the environments with initial state s and $n_s = |I_s|$. For policy z, the basic exported numerator for the first price at node r is

$$A^{z,p1}_{s,r} = \sum_{i \in I_s} w_r p^z_{1,ir}, \qquad A^{weight}_{s,r} = n_s w_r.$$

The plotted mean at that node is $A^{z,p1}_{s,r}/A^{weight}_{s,r}$. The weights cancel because the same node is evaluated in every environment; the averaging is across environments.

Overall expected revenue and information gains are

$$R^z_{s,total} = \sum_{i \in I_s} \sum_r w_r R^z_{ir}, \tag{23}$$

$$\bar{R}^z_s = R^z_{s,total}/n_s, \tag{24}$$

$$\Delta \bar{R}_s = (R^F_{s,total} - R^L_{s,total})/n_s, \tag{25}$$

$$g^{\%}_s = 100 \left(\frac{R^F_{s,total}}{R^L_{s,total}} - 1 \right). \tag{26}$$

The exported column named `weight` is a denominator: it sums to n_s across all nodes. It is not always a probability after aggregation.

15.1 Expected terminal prices

For a type and environment,

$$\mathbb{E}[p^z_2 \mid i, r] = s^z_{1,ir} p^z_{2,ir,j} + (1 - s^z_{1,ir}) p^z_{2,ir,0}.$$

A type-group price path averages this expression across nodes in the group and across environments. It is not p_2 for a representative history. The common-price path uses its own induced history probabilities and the same population weights.

15.2 Price changes and ratios

Dollar change uses $p_1 - p_j^0$ or $\mathbb{E}[p_2] - p_j^0$. The average percentage change computes $100(p_1/p_j^0 - 1)$ inside each environment before averaging. It generally differs from $100(\bar{p}_1/\bar{p}_j^0 - 1)$.

The realized-history ratio plot uses

$$\mathbb{E}[p_1/p_2(h_1) \mid h_0 = s, h_1 = b],$$

not a ratio of average prices and not an unconditional average of the two branch ratios.

15.3 Revenue conditional on a purchase history

For a given branch, first-period revenue is p_1 if inside was chosen and zero if outside was chosen. Terminal revenue remains an expectation over the period-two shock. Thus the branch payoff is

$$p_1 \mathbf{1}\{b = j\} + \delta p_2(b) s_{2r,b}(p_2(b)).$$

The runner weights this by the branch probability before aggregating. Summing the inside and outside branch numerators reconstructs total revenue. In the previous full-basket code, first revenue instead used the realized focal count within a basket; the rewritten binary count is exactly zero or one.

16 How the figures mirror supply, and how to interpret them

1. **Optimal prices against γ.** Left: informed p_1 by initial binary history. Right: informed p_2 after inside and outside. Empirical curves average over environments and connect numerical integration nodes; they are model implications, not fitted data points proving a fit.
2. **Price ratios and conditional profit.** Four (h_0, h_1) histories, with the same history order and line styles as supply. At zero cost, revenue equals profit within the exercise.
3. **Type-specific and common paths.** Low/middle/high groups show informed paths; the dashed path shows the common policy. Terminal offers are expectations over the first purchase outcome.
4. **Price changes.** Compare optimal offers with observed posted prices, in dollars. A negative value means a lower offer, not negative revenue.
5. **Revenue levels.** Expected two-period revenue per model consumer. This is not realized sales or actual household revenue.
6. **Information gains.** Informed minus limited actual expected revenue under the same persistent novelty types. Each prior sensitivity is a distinct population experiment.

A difference between empirical initially-inside and initially-outside averages may reflect both the binary history and differences in the environments being averaged. A change in conditional mean terminal price can reflect selection into the branch as well as the individual price rule.

Decision to verify

The source of an economic explanation should be individual price rules and continuation values, not only the sign of an aggregate curve. "Initially outside has a larger gain" and "initially outside generates more revenue" are different claims. Recheck both with the new binary output rather than importing an explanation from the old run.

17 What the earlier results were, and why they need rerunning

The earlier completed run in `new_pricing_res/` has no binary specification tag, records capacity seven, and explicitly identifies a multinomial habit-inclusive demand model. Its solver hash matches the preserved previous source snapshot.

It enumerated all multinomial baskets. With seven opportunities and M available inside categories, the number of baskets is

$$|\mathcal{B}| = \binom{7+M}{M}.$$

That is 120 for three inside categories, 36 for two, and eight for one. The earlier *7/36 mismatched elements* error therefore came from a terminal check over 36 basket states. The rewritten binary terminal array always has two state columns.

The old code updated every flavor's novelty and stock after a basket, allowed different second prices across those baskets, and evaluated revenue for seven choice opportunities. The new code updates only the focal binary history and own stock, has two second prices, and evaluates one opportunity per period. These are substantive model changes.

For reference only, the old fitted-distribution aggregates were:

Initial	Information	Observed price	p_1	$\mathbb{E}[p_2]$	Revenue
Inside	Full	2.399	2.460	2.541	10.889
Inside	Limited	2.399	2.469	2.539	10.873
Outside	Full	2.293	1.597	1.900	4.076
Outside	Limited	2.293	1.729	1.962	3.988

Those generated old gains of approximately 0.15% and 2.21%. **Do not label these as the binary rewrite's gains.** The display units and dynamic transitions changed. The new result importer explicitly rejects these old outputs for the binary presentation.

18 What has been checked, and what remains a choice

The binary mathematical tests check:

- exact agreement with the separate theoretical solver when habit is zero and baseline parameters match;
- exactly two outcomes and two successor histories even with three or five starting flavor categories;
- the outside inclusive-value formula and the two habit transitions;
- equality of regimes when novelty type is certain;
- dense independent first-price searches for both informed and subjective limited objectives;
- persistent-type revenue reconstruction and informed dominance;
- focal-flavor permutation, missing rivals, zero continuation weight, and negative habit preferences.

The server policy checks still reject empirical execution locally. The new binary empirical job has not yet been completed, so its sample reconstruction, resulting prices, aggregate mapping gaps and numerical sensitivity must be checked after the run.

Verified computational property	Not established by that property
Two state columns	That the chosen outside-value mapping is economically correct
Theory limit matches	That transported multinomial coefficients estimate the binary model
Old likelihood reconstructs	That the new binary choice probabilities fit observed data
Informed revenue dominates	That the magnitude is economically large or causal
Quadrature weights sum to one	That nine nodes are sufficient for every target moment
Price derivative residual is small	That every possible mixture maximum is found analytically
Outputs are aggregate-only	That every underlying environment could be reviewed locally

19 Verification checklist before interpreting the rerun

1. The inside alternative is the selected flavor; all other available flavors and nonpurchase form one outside alternative.
2. Initial observed history is collapsed using any focal purchase versus none. Mixed starting baskets are handled by that rule.
3. There is one binary choice in each counterfactual period, not seven weekly opportunities.
4. The outside baseline uses known observed rival prices and initial rival habit, remains fixed, and excludes novelty because switching is represented separately.
5. Novelty follows the theory's symmetric $\pm\gamma$ effect. Its numerical coefficient is transported directly from the saved fit.
6. Own habit is cumulative, starts from the saved state, and increments only after the inside choice.
7. Price sensitivity and habit are known estimated posterior means in both regimes.
8. Perfect information reveals each population integration type; limited information keeps fixed population weights.
9. The limited first-price objective is the fixed-belief objective, not the Bayesian optimum and not a separately optimized persistent-type first objective.
10. Reported revenue uses the same consumer type through both periods.
11. Common prices can differ across histories and known environments, but not across hidden novelty types within one such environment.
12. Old full-basket results are kept separate. Read the new specification tag and one-opportunity capacity before importing figures.

20 How to run all three flavors and update plain in the presentation

After syncing the changed source to the research server, submit using the directory containing `fit.json` and `point_estimates.csv`:

```
cd /scratch/dtm63837/Kilts_Panel/LOV_CC_2YP
sbatch Code/final_simple/server/run_information_pricing.sh \
  /scratch/dtm63837/Kilts_Panel/rebuilds/run_JOB_ID/fits/no_control all
```

Replace the example fit directory with the actual saved fit directory. For an after-data run it can be a path ending in `three/fits/no_control`. No demand refit is performed. The job reconstructs the fit once, then runs other yogurt, berry and plain as the focal flavor, with separate outputs.

The result directory is

```
/scratch/dtm63837/Kilts_Panel/results/information_pricing_JOB_ID/RESULTS_ONLY/
```

Its summary must contain `specification="binary_focal_vs_not"`, `capacity=1`, and the explicit outside and habit mapping. The fitted estimation capacity is reported separately. Recover only this aggregate-only directory.

To update the plain slides from those new aggregates:

```
python3 Code/final_draft/seminar_pres/refresh_information_pricing.py \
  --source /path/to/new/RESULTS_ONLY --flavor plain
```

Berry and other results remain separate and do not replace the plain main-deck figures. The importer rejects the previous multinomial outputs. It writes to a separate binary asset directory, so old full-basket figures cannot accidentally activate the binary result slides.

To check mathematical code without empirical data:

```
python3 Code/final_simple/checks/check_binary_information_pricing.py
python3 Code/privacy/check_policy.py
```

21 Code reading map and frozen sources

The full binary solver, server orchestration, plotting, presentation import and submission launcher are reproduced below. The reused optimizer, posterior and data reconstruction functions are included with their original line numbers. Optional preprocessing branches are visible even when the saved fit does not use them. No raw-data contents are included.

Filesystem validation, directory creation and JSON/CSV serialization have no economic equation; their syntax is explained in the server-loop map. Statistical estimation optimization is not rerun by this job and is therefore outside its active execution path. The earlier full-basket solver is preserved separately at the end so its additional states can be inspected directly.

Snapshot	Lines	Source
binary_model	97	`Code/final_simple/demand/binary_information_pricing_model.py`
runner	265	`Code/final_simple/demand/information_pricing.py`
optimizer	242	`Code/final_simple/demand/information_pricing_model.py`
history	49	`Code/final_simple/demand/history.py`

posterior	696	`Code/final_simple/demand/split_pricing.py`
data	1359	`Code/final_simple/demand/estimation.py`
categories	57	`Code/final_simple/flavor_specs.py`
policy	53	`Code/privacy/server_data_policy.py`
plots	157	`Code/final_simple/demand/plot_information_pricing.py`
refresh	67	`Code/final_draft/seminar_pres/refresh_information_pricing.py`
theory	163	`Code/final_draft/writeup/1108_writeup/theory/gamma_uncertainty.py`
theory_helpers	215	`Code/final_simple/supply/pricing_model.py`
tests	103	`Code/final_simple/checks/check_binary_information_pricing.py`
launcher	31	`Code/final_simple/server/run_information_pricing.sh`

A Complete binary solver

Source: `Code/final_simple/demand/binary_information_pricing_model.py`.

SHA-256: `c2ecece0ce671ce04764d6522d77c56ac563053e6858dc811e0c83153e013740`.

```python
"""Two binary choices: focal flavor j versus the composite not-j option.

No baskets or rival-specific future histories. Existing fitted coefficients are
plugged into an explicitly specified binary extension of the supply model.
This is a counterfactual parameter mapping, not a refit or an exact dynamic
aggregation of the fitted multinomial model. No empirical I/O occurs here.
"""
import numpy as np
from scipy.special import expit, logsumexp, wrightomega
try:
    from .information_pricing_model import _terminal_common
except ImportError:
    from information_pricing_model import _terminal_common


def solve_binary(prices, available, beta, stock, alpha, habit, gamma, weights,
                 initial_inside, focal=1, delta=1., controls=None, search_points=65):
    """Known alpha/habit, known initial binary history, gamma known or hidden.

    Theory-style relative utility in BOTH periods:
      v_r(p,h,H) = b + gamma_r*(1{h=0}-1{h=j}) + habit*H - alpha*p.
    b = beta_j + fixed control_j - C, where C is the logsumexp of available
    rivals' baseline utilities (including their initial habit stocks) and zero
    for nonpurchase. C, rival prices, and rival stocks stay fixed for two periods.
    gamma is excluded from C because its switching effect is represented by the
    explicit binary history term. Own habit updates H' = H + 1{focal chosen}.
    Initial H comes from the saved cumulative-stock fit. Future periods each have
    ONE choice opportunity, not seven weekly purchase counts.

    Limited information keeps gamma weights fixed at both decisions. Both
    policies are evaluated with the same persistent type through both periods.
    """
    prices, beta, stock, gamma, weights = [np.asarray(x, float) for x in
                                           [prices, beta, stock, gamma, weights]]
    available = np.asarray(available, bool)
    j = len(prices)
    if (any(x.shape != (j,) for x in [beta, stock, available])
            or gamma.ndim != 1 or weights.shape != gamma.shape or not len(gamma)
            or not all(np.isfinite(x).all() for x in [prices, beta, stock, gamma, weights])
            or not np.isfinite([alpha, habit, delta]).all() or alpha <= 0
            or not 0 <= delta <= 1 or (weights < 0).any()
            or not np.isclose(weights.sum(), 1.) or (prices < 0).any()
            or not 0 <= focal < j or not available[focal]
            or initial_inside not in [False, True] or search_points < 17):
        raise ValueError('Invalid binary-pricing arrays, weights, menu or parameters.')
    controls = np.zeros(j) if controls is None else np.asarray(controls, float)
    if controls.shape != (j,) or not np.isfinite(controls).all():
        raise ValueError('Invalid fixed utility controls.')
    rivals = np.flatnonzero(available & (np.arange(j) != focal))
    rival_base = beta[rivals]-alpha*prices[rivals]+habit*stock[rivals]+controls[rivals]
    outside_index = float(logsumexp(np.r_[rival_base, 0.]))
    intercept = float(beta[focal]+controls[focal]-outside_index)
    own_stock = float(stock[focal])
    history_sign = -1. if initial_inside else 1.
    intercept1 = intercept+gamma*history_sign+habit*own_stock
    # Exactly two successor histories, ordered [focal, not focal].
    intercept2 = np.column_stack([intercept-gamma+habit*(own_stock+1.),
                                  intercept+gamma+habit*own_stock])
    p2_full = (1+wrightomega(intercept2-1))/alpha
    r2_full = p2_full*expit(intercept2-alpha*p2_full)
    # Effective-cost transformation solves the informed first price analytically.
    effective_cost = -delta*(r2_full[:, 0]-r2_full[:, 1])
    p1_full = effective_cost+(1+wrightomega(intercept1-alpha*effective_cost-1))/alpha
    # The same fixed gamma weights enter each common terminal optimization.
```

```
    p2_limited, perceived2 = _terminal_common(intercept2, weights, alpha, 1, search_points)
    adjustment = delta*(perceived2[0]-perceived2[1])
    # Subjective value = Q(p)*(p+adjustment)+delta*perceived2[1].
    relative_price, relative_value = _terminal_common(
        intercept1[:, None]+alpha*adjustment, weights, alpha, 1, search_points)
    p1_limited = np.full(len(gamma), relative_price[0]-adjustment)
    limited_objective = float(relative_value[0]+delta*perceived2[1])
    r2_limited = p2_limited[None, :]*expit(intercept2-alpha*p2_limited)
    def evaluate(p1, p2, r2):
        s1 = expit(intercept1-alpha*p1)
        mass = np.column_stack([s1, 1-s1])
        revenue1 = p1*s1
        revenue2 = (mass*r2).sum(axis=1)
        return dict(p1=p1, expected_p2=(mass*p2).sum(axis=1),
                    revenue1=revenue1, revenue2=revenue2,
                    revenue=revenue1+delta*revenue2, state_probability=mass)
    full = evaluate(p1_full, p2_full, r2_full)
    limited = evaluate(p1_limited, np.broadcast_to(p2_limited, p2_full.shape), r2_limited)
    if (full['revenue']+1e-8 < limited['revenue']).any():
        raise AssertionError('Binary informed optimum does not dominate common policy.')
    for result in [full, limited]:
        np.testing.assert_allclose(result['state_probability'].sum(axis=1), 1., atol=1e-12)
    return dict(full=full, limited=limited, gamma=gamma, weights=weights,
                states=np.array([[1, 0], [0, 1]], dtype=int), focal_index=0,
                p2_full=p2_full, p2_limited=p2_limited,
                terminal_revenue_full=r2_full, terminal_revenue_limited=r2_limited,
                limited_beliefs=np.broadcast_to(weights[:, None], (len(gamma), 2)).copy(),
                limited_objective=limited_objective,
                full_revenue=float(weights @ full['revenue']),
                limited_revenue=float(weights @ limited['revenue']),
                gain=float(weights @ (full['revenue']-limited['revenue'])),
                outside_index=outside_index, binary_intercept=intercept,
                first_utility_intercept=intercept1, terminal_utility_intercept=intercept2)
```

B Reused mixture-price optimizer

Source: `Code/final_simple/demand/information_pricing_model.py`.

SHA-256: `9253725cc805537e8373341e725c8db4a733ecf933df50ccbb01ede78b5e4c8d`.

B.1 `_terminal_common`

```
def _terminal_common(intercepts, weights, alpha, capacity, points):
    """Fixed-belief terminal optimum for every basket, shared across novelty types.

    Optimize in t=alpha*p. Every mixture optimum lies between the component
    optima. Grid-screen all local maxima and bisect all positive-to-negative FOCs.
    """
    component = 1+wrightomega(intercepts-1)
    lo, hi = component.min(axis=0), component.max(axis=0)
    count = max(points, int(8*np.max(hi-lo))+1)
    if count > 32769:
        raise ValueError('Terminal mixture too wide for the price search.')
    grid = lo[None, :]+np.linspace(0, 1, count)[:, None]*(hi-lo)[None, :]
    def objective(t, states):
        q = expit(intercepts[:, states]-t[None, :])
        return capacity/alpha*t*(weights[:, None]*q).sum(axis=0)
    def slope(t, states):
        q = expit(intercepts[:, states]-t[None, :])
        return (weights[:, None]*q*(1-t[None, :]*(1-q))).sum(axis=0)
    q = expit(intercepts[:, None, :]-grid[None, :, :])
    d = (weights[:, None, None]*q*(1-grid[None, :, :]*(1-q))).sum(axis=0)
    values = capacity/alpha*grid*(weights[:, None, None]*q).sum(axis=0)
    selected = values.argmax(axis=0)
    states_all = np.arange(len(lo))
    price_t = grid[selected, states_all].copy()
    best = values[selected, states_all].copy()
    rows, states = np.where((d[:-1] >= 0) & (d[1:] <= 0))
    left, right = grid[rows, states], grid[rows+1, states]
    for _ in range(48):
        mid = (left+right)/2
        positive = slope(mid, states) > 0
        left, right = np.where(positive, mid, left), np.where(positive, right, mid)
    roots = (left+right)/2
    root_values = objective(roots, states)
    for state, root, value in zip(states, roots, root_values):
        # Prefer the refined root to a revenue-equivalent grid point. A strict
        # comparison can retain the grid point solely because of rounding.
        tolerance = 64*np.finfo(float).eps*max(abs(best[state]), abs(value),
                                               np.finfo(float).tiny)
        if value >= best[state]-tolerance:
            best[state], price_t[state] = value, root
    np.testing.assert_allclose(slope(price_t, states_all), 0., atol=2e-9)
    return price_t/alpha, best
```

C Complete server orchestration and aggregation

Source: `Code/final_simple/demand/information_pricing.py`.

SHA-256: `3ead24816f5ceec5b397f48054ddd9c8595f53128e968b99823eec63c843cd7c`.

```
"""Server-only BINARY history-adaptive prices; only novelty preference is hidden.

Reuses an existing converged habit-inclusive fit. Writes aggregate results only.
No demand refit, binary price calibration, or individual policy export.
"""
from pathlib import Path
import argparse
import hashlib
import json
import sys
from types import SimpleNamespace

ROOT = next(p for p in Path(__file__).resolve().parents
            if (p/'Code/privacy/server_data_policy.py').is_file())
sys.path.insert(0, str(ROOT/'Code/privacy'))
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from server_data_policy import server_only, require_data_path, require_server
import numpy as np
import pandas as pd
from scipy.special import expit, logsumexp
from numpy.polynomial.hermite import hermgauss
try:
    from . import estimation, split_pricing
    from .history import saved_stock_measure
    from .binary_information_pricing_model import solve_binary
except ImportError:
    import estimation, split_pricing
    from history import saved_stock_measure
    from binary_information_pricing_model import solve_binary
from flavor_specs import categories


def _hash(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


@server_only
def run(args):
    """Reconstruct and verify saved demand, then export aggregate policy comparisons."""
    require_data_path(args.fit_dir)
    require_data_path(args.output)
    if args.output.exists():
        raise FileExistsError('Use a new output directory; existing results are preserved.')
    if args.nodes < 3 or args.nodes > 41 or args.search_points < 17:
        raise ValueError('Use 3..41 quadrature nodes and at least 17 search points.')
    if not 0 <= args.delta <= 1 or (args.max_environments is not None and args.max_environments < 1):
        raise ValueError('Check revenue discount factor and environment limit.')
    fit = json.loads((args.fit_dir/'fit.json').read_text())
    if fit['model'] != 'heterogeneous_history' or not fit.get('converged', False):
        raise ValueError('A converged heterogeneous-history fit is required.')
    settings = dict(fit['settings'])
    if settings.get('novelty_measure', 'previous_share') != 'previous_share':
        raise ValueError('This transition requires the saved previous-share novelty definition.')
    measure = saved_stock_measure(settings)
    if measure != 'cumulative':
        raise ValueError('The binary specification currently requires the saved cumulative-stock fit; no silent
     history conversion.')
    settings['stock_measure'] = measure
    settings.setdefault('stock_retention', 1.)
    settings.setdefault('flavor_spec', 'three')
    settings.setdefault('novelty_measure', 'previous_share')
    for key in ['sample', 'flavors', 'products_extra']:
        settings[key] = Path(settings[key]) if settings.get(key) is not None else None
    for key in ['sample', 'flavors']:
```

```
        override = getattr(args, key, None)
        if override is not None:
            require_data_path(override)
            settings[key] = override
    for key in ['sample', 'flavors', 'products_extra']:
        if settings[key] is not None:
            require_data_path(settings[key])
    settings['output'] = args.output
    selected = categories(settings['flavor_spec'])
    estimation.FLAVORS[:] = selected
    split_pricing.FLAVORS[:] = selected
    parameters_table = pd.read_csv(args.fit_dir/'point_estimates.csv')
    if 'at_bound' in parameters_table and parameters_table.at_bound.astype(str).str.lower().eq('true').any():
        raise ValueError('Review fitted boundary parameters before counterfactual pricing.')
    parameters = dict(zip(parameters_table.parameter, parameters_table.estimate))
    if parameters['sd_novelty'] < 0:
        raise ValueError('Invalid fitted novelty dispersion.')
    # Server-only in-memory reconstruction; do not replace any files in fit_dir.
    frame = estimation.prepare_data(SimpleNamespace(**settings))
    control = not settings.get('no_price_control', False)
    if control:
        frame, _ = estimation.add_price_controls(frame)
    if frame.duplicated(['household_code', 'week']).any():
        raise ValueError('Duplicate household/week rows.')
    if len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['households']:
        raise ValueError('Reconstructed sample totals do not match saved fit.')
    draws = split_pricing.persistent_draws(settings['draws'], settings['seed'])
    households, _, _, _, beta, nll = split_pricing.posterior(frame, parameters, draws, control)
    if not np.isclose(nll, fit['nll'], rtol=1e-9, atol=1e-4):
        raise ValueError('Reconstructed likelihood differs from saved fit; no pricing results written.')
    context = split_pricing.forecast_context(frame, households, settings['stock_retention'], measure)
    first = frame.sort_values(['household_code', 'week']).groupby('household_code', sort=True).head(1)
    first = first.set_index('household_code').loc[households.household_code]
    nodes, weights = hermgauss(args.nodes)
    gamma = parameters['mean_novelty']+np.sqrt(2)*parameters['sd_novelty']*nodes
    weights = weights/np.sqrt(np.pi)
    # Cross the fitted population novelty distribution with each fixed environment.
    # Only alpha/habit are treated as known posterior means, in BOTH regimes.
    cutoff = .6744897501960817*parameters['sd_novelty']
    labels = np.where(gamma < parameters['mean_novelty']-cutoff, 'low',
                      np.where(gamma > parameters['mean_novelty']+cutoff, 'high', 'middle'))
    scenarios = [('Fitted distribution', weights)]
    if all(np.any(labels == group) for group in ['low', 'middle', 'high']):
        for favored in ['low', 'middle', 'high']:
            target = {'low': .25, 'middle': .25, 'high': .25}
            target[favored] = .5
            alternative = weights.copy()
            for group in target:
                mask = labels == group
                alternative[mask] *= target[group]/weights[mask].sum()
            scenarios.append((favored.title()+' type more likely', alternative))
    flavors = list(selected) if args.flavor == 'all' else (['plain', 'berry'] if args.flavor == 'both' else [args.
     flavor])
    if not set(flavors) <= set(selected):
        raise ValueError('Requested focal flavor is absent from saved fit.')
    estimation_capacity = settings['capacity']
    if not isinstance(estimation_capacity, int) or not 1 <= estimation_capacity <= 12:
        raise ValueError('Saved estimation capacity must be an integer in 1..12.')
    args.output.mkdir(parents=True)
    reports = {}
    for flavor in flavors:
        focal = selected.index(flavor)
        eligible = np.flatnonzero(context['available'][:, focal])
        if args.max_environments is not None:
            eligible = eligible[:args.max_environments]
        if not len(eligible):
            raise ValueError('No eligible pricing environments for '+flavor)
        sums = {}
        total_environments = {'inside': 0, 'outside': 0}
        max_mass_error = 0.
        for number, index in enumerate(eligible, 1):
```

```
            state = 'inside' if first.iloc[index][flavor] > 0 else 'outside'
            total_environments[state] += 1
            posted = context['prices'][index, focal]
            for scenario, prior in scenarios:
                known_alpha = float(households.iloc[index].alpha_mean)
                known_habit = float(households.iloc[index].habit_mean)
                fixed_controls = parameters.get('price_control', 0.)*context['cf'][index]
                result = solve_binary(
                    prices=context['prices'][index], available=context['available'][index], beta=beta,
                    stock=context['stocks'][index], alpha=known_alpha, habit=known_habit,
                    gamma=gamma, weights=prior, initial_inside=(state == 'inside'),
                    focal=focal, delta=args.delta, controls=fixed_controls,
                    search_points=args.search_points)
                # Expose the plug-in mapping difference using aggregates only.
                full_utility = (beta-known_alpha*context['prices'][index]
                                +gamma[:, None]*context['novelty'][index]
                                +known_habit*context['stocks'][index]+fixed_controls)
                full_utility = np.where(context['available'][index], full_utility, -np.inf)
                full_utility = np.column_stack([full_utility, np.zeros(len(gamma))])
                posted_multinomial = np.exp(full_utility[:, focal]-logsumexp(full_utility, axis=1))
                posted_binary = expit(result['first_utility_intercept']-known_alpha*posted)
                for regime in ['full', 'limited']:
                    values = result[regime]
                    max_mass_error = max(max_mass_error,
                        float(np.max(np.abs(values['state_probability'].sum(axis=1)-1))))
                    for k, (g, weight, group) in enumerate(zip(gamma, prior, labels)):
                        key = (scenario, state, regime, k, group)
                        metrics = dict(weight=weight, posted=weight*posted,
                            p1=weight*values['p1'][k], expected_p2=weight*values['expected_p2'][k],
                            change_p1=weight*(values['p1'][k]-posted),
                            change_p2=weight*(values['expected_p2'][k]-posted),
                            pct_change_p1=weight*100*(values['p1'][k]/posted-1),
                            pct_change_p2=weight*100*(values['expected_p2'][k]/posted-1),
                            revenue1=weight*values['revenue1'][k], revenue2=weight*values['revenue2'][k],
                            revenue=weight*values['revenue'][k],
                            posted_probability_binary=weight*posted_binary[k],
                            posted_probability_multinomial=weight*posted_multinomial[k],
                            posted_probability_gap=weight*(posted_binary[k]-posted_multinomial[k]))
                        p2 = result['p2_full'][k] if regime == 'full' else result['p2_limited']
                        terminal_revenue = result['terminal_revenue_'+regime][k]
                        mass = values['state_probability'][k]
                        for branch, mask in [('inside', result['states'][:, result['focal_index']] > 0),
                                             ('outside', result['states'][:, result['focal_index']] == 0)]:
                            metrics['mass_'+branch] = weight*mass[mask].sum()
                            metrics['p2_'+branch] = weight*(mass[mask] @ p2[mask])
                            realized = (result['states'][mask, result['focal_index']]*values['p1'][k]
                                        +args.delta*terminal_revenue[mask])
                            metrics['profit_'+branch] = weight*(mass[mask] @ realized)
                            metrics['ratio_'+branch] = weight*(mass[mask] @ (values['p1'][k]/p2[mask]))
                        record = sums.setdefault(key, {name: 0. for name in metrics})
                        for name, value in metrics.items():
                            record[name] += float(value)
            if number % 100 == 0 or number == len(eligible):
                print(f'{flavor}: solved {number}/{len(eligible)} environments', flush=True)
    records = [dict(scenario=scenario, initial_state=state, regime=regime, node=k, group=group,
                    gamma=float(gamma[k]), **values)
               for (scenario, state, regime, k, group), values in sums.items()]
    node_totals = pd.DataFrame(records)
    metric_columns = [c for c in node_totals if c not in ['scenario', 'initial_state', 'regime', 'node', 'group',
 'gamma']]
    destination = args.output/flavor
    destination.mkdir()
    node_totals.to_csv(destination/'type_node_aggregate.csv', index=False)
    by_type = node_totals.groupby(['scenario', 'initial_state', 'regime', 'group'], as_index=False)[
metric_columns].sum()
    by_type.to_csv(destination/'price_paths_by_type.csv', index=False)
    totals = node_totals.groupby(['scenario', 'initial_state', 'regime'], as_index=False)[metric_columns].sum()
    totals.to_csv(destination/'revenue_comparison.csv', index=False)
    comparison = totals.pivot(index=['scenario', 'initial_state'], columns='regime', values='revenue')
    comparison['gain'] = comparison['full']-comparison['limited']
    comparison['gain_pct'] = 100*comparison['gain']/comparison['limited'].where(comparison['limited'] > 0)
```

```
        comparison.to_csv(destination/'information_gain.csv')
        reports[flavor] = dict(environments=len(eligible), initial_states=total_environments,
                               max_mass_error=max_mass_error)
    summary = dict(status='pricing_complete', model='binary habit-inclusive pricing extension',
        specification='binary_focal_vs_not',
        parameter_mapping='plug-in from saved multinomial fit; not an exact dynamic aggregation or binary refit',
        demand_refitted=False, saved_fit_nll=float(fit['nll']), reconstructed_nll=float(nll),
        fit_sha256=_hash(args.fit_dir/'fit.json'),
        parameters_sha256=_hash(args.fit_dir/'point_estimates.csv'),
        solver_sha256=_hash(Path(__file__).with_name('binary_information_pricing_model.py')),
        optimizer_sha256=_hash(Path(__file__).with_name('information_pricing_model.py')),
        flavor_categories=selected, focal_flavors=flavors, stock_measure=measure, stock_retention=settings['stock_retention'],
        novelty_measure='theory binary switching sign: +gamma after not-plain, -gamma after plain',
        estimation_novelty_measure=settings['novelty_measure'],
        capacity=1, estimation_capacity=estimation_capacity, delta=args.delta,
        hidden='novelty preference only', known='price sensitivity and habit: estimated posterior means; histories and menus',
        beliefs='fitted population normal novelty distribution; fixed in every state',
        gamma_nodes=gamma.tolist(), gamma_weights=weights.tolist(), quadrature_nodes=args.nodes,
        prior_scenarios=[dict(name=name, weights=w.tolist()) for name, w in scenarios],
        evaluation='same persistent gamma types under both policies',
        limited_objective='fixed-belief objective at each decision, matching theory; realized revenue accounts for persistent types',
        common_prices='common across novelty types conditional on known alpha, habit, history, rivals and menus',
        initial_history='inside if any focal flavor in first retained purchase week, otherwise outside',
        subsequent_histories='exactly two: focal and not-focal; one binary decision per pricing period',
        habit_transition='H_focal next = H_focal current + indicator(focal chosen); cumulative',
        outside_index='fixed logsumexp of rival baseline utilities including initial rival habit, and zero nonpurchase',
        outside_switching='outside index excludes gamma; switching enters via the explicit binary history sign',
        population='fitted population novelty crossed with observed pricing environments; not realized household revenue',
        participation='focal flavor versus all other available flavors and nonpurchase; outside baseline fixed',
        prices='continuous; first-period prices may rise/fall or be negative; terminal prices nonnegative; zero cost',
        restrictions='all novelty types optimized; no middle-group exclusion',
        search_points=args.search_points, development_limit=args.max_environments,
        results=reports)
    (args.output/'information_pricing_summary.json').write_text(json.dumps(summary, indent=2)+'\n')
    try:
        from .plot_information_pricing import plot_results
    except ImportError:
        from plot_information_pricing import plot_results
    for flavor in flavors:
        plot_results(args.output/flavor, args.output/flavor)
    summary['status'] = 'complete'
    (args.output/'information_pricing_summary.json').write_text(json.dumps(summary, indent=2)+'\n')
    print('Complete. Outputs are aggregate-only; demand fit unchanged.', flush=True)
    return summary


def main():
    require_server()
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--fit-dir', type=Path, required=True)
    parser.add_argument('--output', type=Path, required=True)
    parser.add_argument('--flavor', default='plain')
    parser.add_argument('--sample', type=Path)
    parser.add_argument('--flavors', type=Path)
    parser.add_argument('--nodes', type=int, default=9)
    parser.add_argument('--search-points', type=int, default=65)
    parser.add_argument('--delta', type=float, default=1.)
    parser.add_argument('--max-environments', type=int)
    run(parser.parse_args())


if __name__ == '__main__':
    main()
```

D History rules: only saved cumulative history is accepted by the new binary runner

Source: `Code/final_simple/demand/history.py`.

SHA-256: `21c7b3673c5b636c3f965a72e8f7eb73f306789b82d88ddc7e5a1445d0d8d34e`.

```python
"""Pure calendar-time habit-stock transition, shared by estimation and pricing."""
import numpy as np

DEFAULT_STOCK_RETENTION = 0.9


def advance_stock(stock, share, weeks=1, retention=DEFAULT_STOCK_RETENTION):
    """Pre-choice stock next time: rho**weeks * (current stock + current share).

    Shares arrive at the current observation; unobserved weeks add no purchases.
    Retention one reproduces the historical cumulative sum.
    """
    if not np.isfinite(retention) or not 0 <= retention <= 1:
        raise ValueError("Stock retention must lie in [0, 1].")
    if not np.isfinite(weeks) or weeks <= 0:
        raise ValueError("Stock transitions require a positive calendar-week gap.")
    return retention**weeks * (np.asarray(stock) + np.asarray(share))


DEFAULT_STOCK_MEASURE = "average"


def saved_stock_measure(settings):
    """Missing metadata means the original sum, or the later decay specification."""
    return settings.get("stock_measure", "decaying" if "stock_retention" in settings else "cumulative")


def update_history(stock, share, previous_occasions, weeks=1,
                   measure=DEFAULT_STOCK_MEASURE, retention=DEFAULT_STOCK_RETENTION):
    """Advance a pre-choice history; average counts observed purchase occasions only.

    Zero inside shares leave the average unchanged. Calendar gaps do not enter
    its denominator. Supports basket-by-flavor arrays in pricing.
    """
    if measure == "decaying":
        return advance_stock(stock, share, weeks, retention)
    if measure == "cumulative":
        return np.asarray(stock) + np.asarray(share)
    if measure != "average":
        raise ValueError("Unknown stock measure: " + str(measure))
    n = np.asarray(previous_occasions)
    if not np.isfinite(n).all() or (n < 0).any() or (n != np.floor(n)).any():
        raise ValueError("Previous purchase occasions must be nonnegative integers.")
    share = np.asarray(share)
    purchased = share.sum(axis=-1, keepdims=True) > 0
    denominator = n + purchased
    numerator = np.asarray(stock)*n + share
    return np.divide(numerator, denominator, out=np.zeros_like(numerator, dtype=float),
                     where=denominator > 0)
```

E Reused posterior and starting-context functions

Source: `Code/final_simple/demand/split_pricing.py`.

SHA-256: `f504e82664b1666bed63bf09304ba37548e396d509eb69b7bb32f141d51011d1`.

E.1 logsumexp

```python
def logsumexp(x, axis=None, keepdims=False):
    x = np.asarray(x)
    maximum = np.max(x, axis=axis, keepdims=True)
    result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
    return result if keepdims else np.squeeze(result, axis=axis)
```

E.2 persistent_draws

```python
def persistent_draws(number, seed):
    if number < 2 or number % 2:
        raise ValueError("Draw count must be even and >=2.")
    half = np.random.default_rng(seed).standard_normal((number//2, 3))
    return np.vstack([half, -half])
```

E.3 posterior

```python
def posterior(frame, parameters, draws, control=True):
    """Household-integrated multinomial likelihood; types persist across weeks."""
    labels = np.sort(frame.household_code.unique())
    hh = pd.Categorical(frame.household_code, categories=labels).codes
    prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
    available = np.isfinite(prices) & (prices > 0)
    counts = frame[FLAVORS+["outside"]].to_numpy(float)
    novelty = frame[["novelty_"+f for f in FLAVORS]].to_numpy(float)
    stock = frame[["stock_"+f for f in FLAVORS]].to_numpy(float)
    cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
    if (not np.isfinite(counts).all() or (counts < 0).any()
            or not np.isfinite(novelty).all() or not np.isfinite(stock).all() or not np.isfinite(cf).all()
            or ((counts[:, :len(FLAVORS)] > 0) & ~available).any()):
        raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
    beta = parameters["inside_constant"] + np.array([0.] + [parameters["beta_"+f] for f in FLAVORS[1:]])
    alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
    switching = parameters["mean_novelty"] + parameters["sd_novelty"]*draws[:, 1]
    habit = parameters["mean_own_stock"] + parameters["sd_own_stock"]*draws[:, 2]
    loglike = np.empty((len(labels), len(draws)))
    for d, (a, g, b) in enumerate(zip(alpha, switching, habit)):
        u = beta-a*np.nan_to_num(prices)+g*novelty+b*stock+parameters.get("price_control", 0.)*cf
        u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
        logp = u-logsumexp(u, axis=1, keepdims=True)
        ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
        loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
    weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
    nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
    means = weights @ switching
    # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
    order = np.argsort(means, kind="stable")
    groups = np.full(len(labels), "middle", dtype=object)
    n = len(labels)//4
    if n < 1:
        raise ValueError("Need at least four households to form quartiles.")
    groups[order[:n]], groups[order[-n:]] = "low", "high"
    population_q = np.quantile(switching, [.25, .75])
    table = pd.DataFrame(dict(household_code=labels, group=groups, switching_mean=means,
```

```
                            switching_sd=np.sqrt(np.maximum(weights @ switching**2-means**2, 0)),
                            alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
                            probability_switching_below_draw_q25=weights @ (switching <= population_q[0]),
                            probability_switching_above_draw_q75=weights @ (switching >= population_q[1])))
    table["habit_mean"] = weights @ habit
    table["habit_sd"] = np.sqrt(np.maximum(weights @ habit**2 - table.habit_mean.to_numpy()**2, 0))
    return table, weights, alpha, np.column_stack([switching, habit]), beta, nll
```

E.4 `forecast_context`

```
def forecast_context(frame, households, stock_retention=DEFAULT_STOCK_RETENTION,
                     stock_measure=DEFAULT_STOCK_MEASURE):
    """First observed choice establishes h0; price the next two consecutive weeks.

    Carry the initial observation's menu, posted prices and controls forward.
    Type estimation still uses the complete observed sample.
    """
    first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
    first = first.set_index("household_code").loc[households.household_code]
    counts = first[FLAVORS].to_numpy(float)
    totals = counts.sum(axis=1, keepdims=True)
    shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
    # Advance the pre-choice stock to the next calendar week.
    previous_occasions = (first["stock_occasions"].to_numpy(int)[:, None]
                          if "stock_occasions" in first else np.zeros((len(first), 1), dtype=int))
    stocks = update_history(first[["stock_"+f for f in FLAVORS]].to_numpy(float),
                            shares, previous_occasions,
                            measure=stock_measure, retention=stock_retention)
    occasions = previous_occasions + (totals > 0)
    novelty = shares.sum(axis=1, keepdims=True)-shares
    prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
    available = np.isfinite(prices) & (prices > 0)
    eligible = available[:, 1]
    households["berry_eligible"] = eligible
    initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
    households["initial_observed_week"] = initial_weeks
    households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
    households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
    cf_cols = ["cf_"+f for f in FLAVORS]
    cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
    if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
        raise ValueError("Nonfinite forecast history.")
    return dict(stocks=stocks, stock_retention=stock_retention,
                stock_measure=stock_measure, stock_occasions=occasions, novelty=novelty, prices=np.where(available,
     prices, 0.),
                available=available, cf=cf, eligible=eligible)
```

F Saved-data reconstruction dependencies

Source: `Code/final_simple/demand/estimation.py`.

SHA-256: `c5fa94767abcce080222a172f85175390f49aa445b5e90893a7ba0c5b2154d92`.

F.1 `normalized_upc_expr`

```
def normalized_upc_expr():
    text = pl.col("upc").cast(pl.String).str.strip_chars()
    numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
    canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
    return (
        pl.when(numeric)
        .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
        .otherwise(text)
        .alias("upc")
    )
```

F.2 `read_flavors`

```
@server_only
def read_flavors(path, products_extra=None):
    """Use the same plain-first classification as data_code/summ_stats.py."""
    frame = pd.read_csv(path, dtype={"upc": str})
    if {"upc", "flavor"}.issubset(frame.columns):
        # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
        frame["upc"] = normalize_upcs(frame["upc"])
        if (
            frame["upc"].isna().any()
            or frame["upc"].eq("").any()
            or frame["upc"].duplicated().any()
        ):
            raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
        if not frame["flavor"].isin(FLAVORS).all():
            raise ValueError(f"{path}: saved mapping must use {FLAVORS}.")
        print(
            f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
        )
        return frame[["upc", "flavor"]]
    missing = {"upc", "flavor_descr"} - set(frame.columns)
    if missing:
        raise ValueError(f"{path}: missing columns {sorted(missing)}")
    frame["upc"] = normalize_upcs(frame["upc"])
    if frame["upc"].isna().any() or frame["upc"].eq("").any():
        raise ValueError(f"{path}: flavor reference contains missing UPCs.")
    if products_extra is not None:
        products_extra = Path(products_extra)
        if products_extra.exists():
            products = pl.scan_parquet(products_extra)
            names = products.collect_schema().names()
            if not {"upc", "flavor_descr"}.issubset(names):
                raise ValueError(
                    f"{products_extra}: needs upc and flavor_descr columns."
                )
            columns = [
                c
                for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
                if c in names
            ]
            products = (
                products.select(columns)
                .with_columns(normalized_upc_expr())
                .join(
```

```
                    pl.DataFrame(
                        {"upc": frame["upc"].dropna().unique().tolist()}
                    ).lazy(),
                    on="upc",
                    how="anti",
                )
            )
            products = products.filter(
                pl.col("upc").is_not_null()
                & pl.col("flavor_descr").is_not_null()
                & ~pl.col("flavor_descr")
                .cast(pl.String)
                .str.strip_chars()
                .str.to_lowercase()
                .is_in(["", "na", "n/a", "nan", "null", "none"])
            )
            if "panel_year" in names:
                products = products.group_by("upc").agg(
                    [
                        pl.col(c)
                        .sort_by("panel_year", nulls_last=True)
                        .first()
                        .alias(c)
                        for c in columns
                        if c != "upc"
                    ]
                )
            else:
                products = products.unique(
                    subset="upc", keep="first", maintain_order=True
                )
            recovered = products.collect(engine="streaming").to_pandas()
            frame = pd.concat([frame, recovered], ignore_index=True)
            print(
                f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
                flush=True,
            )
        else:
            print(
                f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
                flush=True,
            )
    text = frame["flavor_descr"].fillna("")
    if "product_descr" in frame.columns:
        text = text + " " + frame["product_descr"].fillna("")
    text = text.str.lower()
    from flavor_specs import classify_descriptor
    specification = "five" if len(FLAVORS) == 5 else "three"
    frame["flavor"] = text.map(lambda value: classify_descriptor(value, specification))
    # Match summ_stats: retain missing labels and use the first CSV row per UPC.
    conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
    if conflicts:
        print(
            f"{conflicts:,} UPCs have conflicting flavor classifications; "
            "keeping the first flavor in CSV row order.",
            flush=True,
        )
    return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

F.3 prepare_data

```
@server_only
def prepare_data(settings):
    """Use only the merged purchase table, including its existing prices.

    The IV pools all flavors: the same-week average observed price outside the
    consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
    """
```

```
    FLAVORS[:] = categories(getattr(settings, "flavor_spec", "three"))
    base = pl.scan_parquet(settings.sample)
    names = base.collect_schema().names()
    if "household_size" not in names:
        raise ValueError("Estimation requires household_size to restrict purchases to single-person households.")
    base = base.filter(pl.col("panel_year").is_between(settings.first_year, settings.last_year))
    if "flavor" not in names:
        products_extra = getattr(settings, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet
     "
        flavor = read_flavors(settings.flavors, products_extra)
        base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
    else:
        base = base.with_columns(normalized_upc_expr())
    price_column = next((c for c in ["price", "price_rms"] if c in names), None)
    if price_column is None:
        raise ValueError("Merged purchase table needs a price or price_rms column.")
    consumer_dma = getattr(settings, "household_dma_column", "dma_cd")
    columns = ["household_code", "household_size", "panel_year", "purchase_date", "trip_code_uc", "upc",
               "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
    if not settings.no_price_control:
        columns.append(pl.col(consumer_dma).alias("dma_cd"))
    base = base.select(columns).with_columns(
        pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
    base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
    base = base.with_columns((pl.col("purchase_date") + pl.duration(
        days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
    # Keep the full merged table for prices/IVs, even in a household-limited run.
    # Restrict demand histories before weekly aggregation; retain all quotes for menus/IVs.
    selected = base.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
    if settings.max_households:
        ids = (selected.select("household_code").unique().sort("household_code")
               .limit(settings.max_households).collect(engine="streaming")["household_code"])
        selected = selected.filter(pl.col("household_code").is_in(ids.implode()))
    print(f"Loading merged purchase table: {settings.sample}", flush=True)
    lines = selected.collect(engine="streaming").to_pandas()
    if lines.empty:
        raise ValueError("No purchase lines in selected sample.")
    if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
        raise ValueError("Merged purchase table has missing or invalid flavor labels.")
    q = lines["quantity"].to_numpy(float)
    if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
        raise ValueError("Quantity must be positive integer package counts.")
    lines["week"] = pd.to_datetime(lines["week"])
    keys = ["household_code", "week"]
    weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
              .reindex(columns=FLAVORS, fill_value=0).reset_index())
    stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
    stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
    stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
    weekly = weekly.merge(stores, on=keys, validate="one_to_one")
    if not settings.no_price_control:
        lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
        if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
            raise ValueError("Multiple consumer DMAs within a household-week.")
        weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")

    quote_keys = ["store_code_uc", "week", "upc"]
    price_columns = quote_keys + ["flavor", "price"]
    if not settings.no_price_control:
        price_columns.append("dma_cd")
    quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
              .select(price_columns).unique())
    # A store/week/UPC should have one price and one flavor. Consumer DMA may
    # differ when consumers from different markets visit the same store.
    local = quotes.select(quote_keys + ["flavor", "price"]).unique()
    conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
                   .limit(1).collect(engine="streaming"))
    if len(conflicting):
        raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
    menus = (local.group_by(["store_code_uc", "week", "flavor"])
             .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
```

```
    menus["week"] = pd.to_datetime(menus["week"])
    menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
             .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
    weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
    print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
    if not settings.no_price_control:
        invalid_dma = quotes.select(
            ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
             | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
             | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64, strict=False).
     floor()))
            .fill_null(True).any()).collect(engine="streaming").item()
        if invalid_dma:
            raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
        # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
        by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
                  .group_by(["week", "dma_cd"]).agg(
                      pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
                  .collect(engine="streaming").to_pandas())
        by_dma["week"] = pd.to_datetime(by_dma["week"])
        totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
                  .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
        weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
        weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
        denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
        weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
                                / denominator.where(denominator > 0))
        for f in FLAVORS:
            weekly["iv_" + f] = weekly["iv_overall"]
        weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
        print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
    weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), settings.habit_weight,
                        getattr(settings, "novelty_measure", "previous_share"),
                        getattr(settings, "stock_retention", DEFAULT_STOCK_RETENTION),
                        getattr(settings, "stock_measure", DEFAULT_STOCK_MEASURE))
    price_cols = ["price_" + f for f in FLAVORS]
    available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
    counts = weekly[FLAVORS].to_numpy(float)
    complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
    for j, f in enumerate(FLAVORS):
        weekly["available_" + f] = available[:, j]
    print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
    if not settings.no_price_control:
        iv_cols = ["iv_" + f for f in FLAVORS]
        iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
        print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True
    )
        complete &= iv_complete
    weekly = weekly.loc[complete].copy()
    repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
    print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated, 'household_code'].nunique()
     :,}", flush=True)
    weekly = weekly.loc[repeated].reset_index(drop=True)
    weekly["outside"] = np.maximum(settings.capacity - weekly[FLAVORS].sum(axis=1), 0)
    if weekly.empty:
        raise ValueError("No households with at least two usable purchase weeks.")
    return weekly
```

F.4 check_dma

```
def check_dma(values, description):
    """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
    codes = pd.to_numeric(values, errors="coerce")
    valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
    if not valid.all():
        raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
    return codes.astype("int64")
```

F.5 switch_away_event

```
def switch_away_event(before, after):
    """Binary recovery event for each flavor; inputs are consecutive basket shares.

    A strict majority is dominance. A new entry only rewards flavors whose
    shares fall. Multiple triggers still produce a single unit boost.
    """
    entered = (before == 0) & (after > 0)
    takeover = ((before > .5)[:, None] & (after > .5)[None, :]
                & ~np.eye(len(before), dtype=bool)).any(axis=1)
    return (takeover | (entered.any() & (after < before))).astype(float)
```

F.6 add_stocks

```
def add_stocks(weekly, habit_weight=0.7, novelty_measure="previous_share",
               stock_retention=DEFAULT_STOCK_RETENTION, stock_measure=DEFAULT_STOCK_MEASURE):
    """Record average purchase-share history and novelty before current purchases.

    previous_share: legacy N = 1 - own share in the preceding calendar week.
    lagged_switch_away: N in t is the binary switch-away event from t-2 to t-1.
    Both transitions must be adjacent calendar weeks; missing history gives 0.
    Average: H_t = mean of all preceding observed purchase-week shares.
    Empty prehistory gives zero; missing calendar weeks add no occasions.
    Decaying and cumulative measures remain available for saved-fit reproduction.
    """
    update_history(np.zeros(len(FLAVORS)), np.zeros(len(FLAVORS)), 0,
                   measure=stock_measure, retention=stock_retention)
    if novelty_measure not in {"previous_share", "lagged_switch_away"}:
        raise ValueError("Unknown novelty measure: " + str(novelty_measure))
    result = (
        weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
    )
    stocks = np.zeros((len(result), len(FLAVORS)))
    novelty = np.zeros_like(stocks)
    history_occasions = np.zeros(len(result), dtype=int)
    for _, group in result.groupby("household_code", sort=False):
        stock = np.zeros(len(FLAVORS))
        occasions = 0
        previous_share = np.zeros(len(FLAVORS))
        previous_week = None
        previous_event = np.zeros(len(FLAVORS))
        for index, row in group.iterrows():
            if previous_week is not None:
                gap = (row["week"] - previous_week).total_seconds() / (7 * 86400)
                stock = update_history(stock, previous_share, occasions, gap,
                                       stock_measure, stock_retention)
                occasions += int(previous_share.sum() > 0)
            stocks[index] = stock
            history_occasions[index] = occasions
            adjacent = previous_week is not None and (row["week"] - previous_week).days == 7
            if adjacent:
                novelty[index] = (previous_share.sum() - previous_share
                                  if novelty_measure == "previous_share" else previous_event)
            counts = row[FLAVORS].to_numpy(float)
            current_share = counts / counts.sum()
            previous_event = (switch_away_event(previous_share, current_share)
                              if adjacent else np.zeros(len(FLAVORS)))
            previous_share = current_share
            previous_week = row["week"]
    if stock_measure == "average":
        result["stock_occasions"] = history_occasions
    for j, name in enumerate(FLAVORS):
        result["stock_" + name] = stocks[:, j]
        result["novelty_" + name] = novelty[:, j]
        result["xi_" + name] = (1 - habit_weight) * novelty[
            :, j
        ] - habit_weight * stocks[:, j]
```

```
543     return result
```

G Flavor ordering and classification definitions

Source: `Code/final_simple/flavor_specs.py`.

SHA-256: `1bd6d783d1d201b250dbf0b7b275a5bc95ac79b2dc8c88b2e4b71c250e72426a`.

```
"""Flavor specifications and descriptor rules. No empirical I/O."""
import re

SPECIFICATIONS = {
    'three': ['other', 'berry', 'plain'],
    'five': ['other', 'berry', 'plain', 'other_fruits', 'desserts'],
}
PLAIN = r'plain|unflavored'
# Preserve the original three-group berry rule exactly.
BERRY = r'berry|straw|blue|rasp|black|cran|cherry|wildberry'
EXPANDED_BERRY = r'berr(?:y|ies)|straw|blue|rasp|cran'
DESSERT = (r'vanilla|chocolat|\bpie\b|cheesecake|cake|cookie|cookies|brownie|'
           r'pudding|custard|caramel|toffee|butterscotch|fudge|tiramisu|'
           r'cobbler|\btart\b|dessert|sundae|ice\s*cream|creme\s*brulee|'
           r'cr[eê]me\s*br[uû]l[eé]e|dulce|marshmallow|praline|'
           r'cinnamon\s*roll|boston\s*cream|banana\s*cream')
OTHER_FRUIT = (r'cherry|cherries|peach|apricot|nectarine|mango|pineapple|banana|\bapple\b|'
               r'\bpear\b|orange|lemon|lime|citrus|grapefruit|tangerine|'
               r'mandarin|clementine|plum|prune|grape|raisin|fig|date|'
               r'coconut|passion\s*fruit|guava|papaya|kiwi|melon|'
               r'watermelon|cantaloupe|honeydew|pomegranate|lychee|'
               r'tropical|mixed\s*fruit|fruit\s*medley')


def categories(specification='three'):
    try:
        return list(SPECIFICATIONS[specification])
    except KeyError:
        raise ValueError('Unknown flavor specification: '+str(specification)) from None


def classify_descriptor(text, specification='three'):
    """Classify a descriptor; dessert beats fruit, plain retains first priority."""
    categories(specification)
    text = str(text or '').lower()
    if re.search(PLAIN, text):
        return 'plain'
    if specification == 'five' and re.search(DESSERT, text):
        return 'desserts'
    if re.search(BERRY if specification == 'three' else EXPANDED_BERRY, text):
        return 'berry'
    if specification == 'five' and re.search(OTHER_FRUIT, text):
        return 'other_fruits'
    return 'other'


def classification_expression(text, specification='three'):
    """Equivalent Polars expression for server-side reference classification."""
    import polars as pl
    categories(specification)
    expression = pl.when(text.str.contains(PLAIN)).then(pl.lit('plain'))
    if specification == 'five':
        expression = expression.when(text.str.contains(DESSERT)).then(pl.lit('desserts'))
    expression = expression.when(text.str.contains(BERRY if specification == 'three' else EXPANDED_BERRY)).then(pl.
     lit('berry'))
    if specification == 'five':
        expression = expression.when(text.str.contains(OTHER_FRUIT)).then(pl.lit('other_fruits'))
    return expression.otherwise(pl.lit('other'))
```

H Complete aggregate figure generation

Source: `Code/final_simple/demand/plot_information_pricing.py`.

SHA-256: `2a76d99a5d3f56773af884819a702ee6e92c32a877ad2cf842f4e98955fc1c75`.

```python
"""Supply-style figures from explicitly named aggregate pricing exports only."""
from pathlib import Path
import argparse
import hashlib
import json
import os
import tempfile

import numpy as np
import pandas as pd
os.environ.setdefault('MPLCONFIGDIR', str(Path(tempfile.gettempdir())/'information-pricing-mpl'))


def plot_results(source, output):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    source, output = Path(source), Path(output)
    names = ['type_node_aggregate.csv', 'price_paths_by_type.csv', 'revenue_comparison.csv']
    nodes, groups, totals = [pd.read_csv(source/name) for name in names]
    for table in [nodes, groups, totals]:
        if any(c in table for c in ['household_code', 'household_id', 'week', 'upc']):
            raise ValueError('Figures accept aggregate result tables only.')
        if not {'scenario', 'initial_state', 'regime', 'weight', 'p1', 'expected_p2', 'revenue'} <= set(table):
            raise ValueError('Missing aggregate pricing columns.')
    metadata_path = source.parent/'information_pricing_summary.json'
    metadata = json.loads(metadata_path.read_text()) if metadata_path.is_file() else {}
    binary = metadata.get('specification') == 'binary_focal_vs_not'
    output.mkdir(parents=True, exist_ok=True)
    base = 'Fitted distribution'
    baseline = nodes[nodes.scenario == base]
    color = {'inside': '#2E6896', 'outside': '#B35C21'}
    policy_colors = {'full': '#2E6896', 'limited': '#B35C21'}
    policy_names = {'full': 'Household-specific prices', 'limited': 'Common prices across $\gamma$'}
    state_labels = {'inside': 'Initially inside', 'outside': 'Initially outside'}
    files = []
    def save(fig, stem):
        for ext in ['pdf', 'png']:
            fig.savefig(output/(stem+'.'+ext), bbox_inches='tight', dpi=180)
            files.append(stem+'.'+ext)
        plt.close(fig)
    # Same layout as theoretical optimal-price plots: p1 by h0, p2 by h1.
    full = baseline[baseline.regime == 'full']
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2), layout='constrained')
    for initial in ['inside', 'outside']:
        x = full[full.initial_state == initial].sort_values('gamma')
        if not x.empty:
            axes[0].plot(x.gamma, x.p1/x.weight, color=color[initial], lw=2, label=state_labels[initial])
    pooled = full.groupby('gamma', as_index=False).sum(numeric_only=True).sort_values('gamma')
    for branch in ['inside', 'outside']:
        axes[1].plot(pooled.gamma, pooled['p2_'+branch]/pooled['mass_'+branch],
                     color=color[branch], lw=2, label='Previous '+branch)
    for ax, title in zip(axes, ['Optimal period-1 prices', 'Optimal period-2 prices']):
        ax.set(xlabel=r'$\gamma$', ylabel='Mean optimal price ($)', title=title)
        ax.legend(fontsize=8);ax.grid(axis='y', alpha=.2)
    save(fig, 'optimal_prices_by_gamma')
    # Four initial/realized-history combinations, matching the supply panels.
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2), layout='constrained')
    # Match the supply exercise's history order, colors and line styles.
    for initial, branch, c, style in [
            ('outside', 'outside', '#1f77b4', '-'),
            ('outside', 'inside', '#ff7f0e', '--'),
            ('inside', 'inside', '#2ca02c', '-.'),
            ('inside', 'outside', '#d62728', ':')]:
```

```
        x = full[full.initial_state == initial].sort_values('gamma')
        if x.empty:
            continue
        h0, h1 = ('j' if initial == 'inside' else '0'), ('j' if branch == 'inside' else '0')
        label = rf'$(h_0,h_1)=({h0},{h1})$'
        axes[0].plot(x.gamma, x['ratio_'+branch]/x['mass_'+branch], color=c, ls=style, lw=2, label=label)
        axes[1].plot(x.gamma, x['profit_'+branch]/x['mass_'+branch], color=c, ls=style, lw=2, label=label)
    for ax, title, ylabel in zip(axes, ['Price ratios by realized history', 'Profit conditional on first purchase outcome'],
                                 [r'Mean $p_1/p_2$', 'Expected two-period revenue ($)']):
        ax.set(xlabel=r'$\gamma$', ylabel=ylabel, title=title)
        ax.legend(fontsize=8);ax.grid(axis='y', alpha=.2)
    save(fig, 'price_ratios_and_profit')
    # Type-specific versus common paths in the two initial-history panels.
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2), sharey=True, layout='constrained')
    palette = {'low': '#2E6896', 'middle': '#208475', 'high': '#B35C21'}
    selected = groups[groups.scenario == base]
    for ax, initial in zip(axes, ['inside', 'outside']):
        x = selected[(selected.initial_state == initial) & (selected.regime == 'full')]
        x = x.assign(_order=x.group.map({'low': 0, 'middle': 1, 'high': 2})).sort_values('_order')
        for row in x.itertuples():
            ax.plot([1, 2], [row.p1/row.weight, row.expected_p2/row.weight], marker='o',
                    lw=2, color=palette[row.group], label=row.group.title()+' type')
        common = totals[(totals.scenario == base) & (totals.initial_state == initial) & (totals.regime == 'limited')]
        if len(common):
            r = common.iloc[0]
            ax.plot([1, 2], [r.p1/r.weight, r.expected_p2/r.weight], marker='D', linestyle='--',
                    color='#232C35', lw=2, label='Common prices across $\gamma$')
        ax.set(title=state_labels[initial], xticks=[1, 2], xticklabels=[r'$p_1$', r'$\mathbb{E}[p_2]$'], ylabel='Mean price ($)')
        ax.legend(fontsize=8);ax.grid(axis='y', alpha=.2)
    save(fig, 'information_price_paths_comparison')
    scenarios = list(dict.fromkeys(totals.scenario))
    for metric in ['revenue', 'gain']:
        fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True, layout='constrained')
        for ax, initial in zip(axes, ['inside', 'outside']):
            x = totals[totals.initial_state == initial]
            if x.empty:
                ax.set_title(state_labels[initial]+' (no eligible environments)')
                continue
            by_case = {regime: x[x.regime == regime].set_index('scenario').reindex(scenarios)
                       for regime in ['full', 'limited']}
            if metric == 'revenue':
                for regime, offset in [('limited', -.18), ('full', .18)]:
                    frame = by_case[regime]
                    ax.bar(np.arange(len(scenarios))+offset, frame.revenue/frame.weight,
                           width=.36, color=policy_colors[regime], label=policy_names[regime])
            else:
                mean_full = by_case['full'].revenue/by_case['full'].weight
                mean_limited = by_case['limited'].revenue/by_case['limited'].weight
                ax.bar(np.arange(len(scenarios)), mean_full-mean_limited,
                       color=policy_colors['full'], label='Household-specific − common')
            ax.set(title=state_labels[initial], xticks=np.arange(len(scenarios)),
                   xticklabels=[s.replace(' distribution', '\ndistribution').replace(' type more likely', '\nmore likely') for s in scenarios])
            ax.legend(fontsize=7);ax.grid(axis='y', alpha=.2)
        axes[0].set_ylabel('Expected two-period revenue ($)' if metric == 'revenue' else 'Revenue gain per model consumer ($)')
        save(fig, 'information_revenues' if metric == 'revenue' else 'information_value')
    # The price question: how far do offers move from observed posted prices?
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2), sharey=True, layout='constrained')
    for ax, initial in zip(axes, ['inside', 'outside']):
        for regime in ['full', 'limited']:
            x = totals[(totals.scenario == base) & (totals.initial_state == initial) & (totals.regime == regime)]
            if x.empty:
                continue
            r = x.iloc[0]
            ax.plot([1, 2], [r.change_p1/r.weight, r.change_p2/r.weight], marker='o', lw=2,
                    color=policy_colors[regime], label=policy_names[regime])
        ax.axhline(0., color='gray', lw=.8)
        ax.set(title=state_labels[initial], xticks=[1, 2], xticklabels=[r'$p_1$', r'$\mathbb{E}[p_2]$'],
```

```
                ylabel='Mean change from observed price ($)')
        ax.legend(fontsize=8);ax.grid(axis='y', alpha=.2)
    save(fig, 'price_changes')
    (output/'figure_provenance.json').write_text(json.dumps(dict(
        source_files={name: hashlib.sha256((source/name).read_bytes()).hexdigest() for name in names},
        figures=files, baseline=base,
        meaning='Means across observed environments; gamma curves connect numerical integration nodes',
        specification=metadata.get('specification', 'previous_full_basket_model'),
        terminal=('Exactly two outcomes: focal flavor and not focal' if binary else
                  'Conditional on at least one focal purchase versus no focal purchase; averages over baskets'),
        ratio='Conditional mean of realized p1/p2, not ratio of population-average prices',
        profit='Expected revenue conditional on purchase branch; zero marginal cost',
        common='Common across novelty types within a known alpha/habit/history environment'), indent=2)+'\n')
    return files


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--source', type=Path, required=True)
    parser.add_argument('--output', type=Path)
    args = parser.parse_args()
    plot_results(args.source, args.output or args.source)


if __name__ == '__main__':
    main()
```

I Complete aggregate import into the presentation

Source: `Code/final_draft/seminar_pres/refresh_information_pricing.py`.

SHA-256: `6572a0c95141341c6f73c0a3163796740de9fea07bbcb1af81d8e7508f3d8652`.

```
"""Import only aggregate revised-pricing results and render supply-style slide assets."""
from pathlib import Path
import argparse
import json
import sys

import pandas as pd

ROOT = next(p for p in Path(__file__).resolve().parents
            if (p/'Code/final_simple/demand/plot_information_pricing.py').is_file())
sys.path.insert(0, str(ROOT/'Code/final_simple'))
from demand.plot_information_pricing import plot_results
HERE = Path(__file__).resolve().parent


def refresh(source, flavor='plain'):
    source = Path(source)
    summary = json.loads((source/'information_pricing_summary.json').read_text())
    if (summary.get('status') != 'complete' or summary.get('demand_refitted') is not False
            or summary.get('hidden') != 'novelty preference only'
            or summary.get('specification') != 'binary_focal_vs_not'
            or summary.get('capacity') != 1
            or summary.get('development_limit') is not None):
        raise ValueError('Need completed binary plain-versus-not results; earlier full-basket outputs cannot populate binary slides.')
    if flavor not in summary['results']:
        raise ValueError('Focal flavor absent from completed run.')
    output = HERE/'assets/binary_information_pricing'/flavor
    plot_results(source/flavor, output)
    totals = pd.read_csv(source/flavor/'revenue_comparison.csv')
    rows = [r'\begin{tabular}{llrrr}\toprule',
            r'Initial state & Prior & Household-specific & Common & Gain\\\midrule']
    baseline_gains = {}
    for scenario in dict.fromkeys(totals.scenario):
        for state in ['inside', 'outside']:
            selected = totals[(totals.scenario == scenario) & (totals.initial_state == state)].set_index('regime')
            if selected.empty:
                continue
            values = selected.revenue/selected.weight
            gain = values['full']-values['limited']
            rows.append(f'{state.title()} & {scenario} & {values["full"]:.3f} & {values["limited"]:.3f} & {gain:.3f} '+r'\\')
            if scenario == 'Fitted distribution':
                baseline_gains[state] = dict(gain=float(gain), pct=float(100*gain/values['limited']))
    rows.append(r'\bottomrule\end{tabular}')
    (output/'revenue_table.tex').write_text('\n'.join(rows)+'\n')
    text = '; '.join(f'Initially {state}: {values["pct"]:.2f}'+r'\%'
                     +f' (\${values["gain"]:.3f} per consumer)'
                     for state, values in baseline_gains.items())
    (output/'gain_headline.tex').write_text(text+'.\n')
    result = summary['results'][flavor]
    inside, outside = (result['initial_states'][key] for key in ['inside', 'outside'])
    (output/'run_context.tex').write_text(
        f'{result["environments"]:,} observed price/history environments '
        +f'({inside:,} initially inside, {outside:,} initially outside).\n')
    allowed = ['status', 'model', 'specification', 'parameter_mapping', 'outside_index', 'habit_transition', 'estimation_capacity', 'demand_refitted', 'fit_sha256', 'parameters_sha256',
               'solver_sha256', 'stock_measure', 'stock_retention', 'novelty_measure',
               'capacity', 'delta', 'hidden', 'known', 'beliefs', 'common_prices',
               'evaluation', 'quadrature_nodes', 'search_points', 'results']
    (output/'run_provenance.json').write_text(json.dumps({k: summary[k] for k in allowed}, indent=2)+'\n')
    print('Revised pricing slide assets ready:', output)


```

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--source', type=Path, required=True, help='Recovered aggregate-only RESULTS_ONLY folder.')
    parser.add_argument('--flavor', default='plain', choices=['plain', 'berry', 'other'])
    args = parser.parse_args()
    refresh(args.source, args.flavor)
```

J Complete server submission launcher

Source: `Code/final_simple/server/run_information_pricing.sh`.

SHA-256: `ab324fb84fd0fd7bc14b689f0315f6f3216b0c7b4915c5650ef5356a5d1ebe5a`.

```
#!/bin/bash
#SBATCH --job-name=novelty_info
#SBATCH --partition=hugemem_p
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=300G
#SBATCH --time=3-00:00:00
#SBATCH --output=/scratch/dtm63837/Kilts_Panel/information_pricing_%j.log
set -euo pipefail
umask 077
server_root=/scratch/dtm63837/Kilts_Panel
project_dir="$server_root/LOV_CC_2YP"
if [[ "$(uname -s)" != Linux || ! -d "$server_root" || -z "${SLURM_JOB_ID:-}" ]]; then
    echo 'Submit with sbatch on the research server.' >&2
    exit 1
fi
if [[ $# -lt 1 || $# -gt 2 ]]; then
    echo "Usage: sbatch $0 EXISTING_FIT_DIRECTORY [plain|berry|other|all]" >&2
    exit 1
fi
fit_dir="$1"
focal_flavor="${2:-plain}"
ml Python/3.12.3-GCCcore-13.3.0
export MPLCONFIGDIR="$server_root/information_pricing_cache/matplotlib"
export PYTHONPYCACHEPREFIX="$server_root/information_pricing_cache/python"
echo "Binary specification: focal flavor versus not focal in both periods; two successor histories."
python3 "$project_dir/Code/final_simple/demand/information_pricing.py" \
    --fit-dir "$fit_dir" \
    --output "$server_root/results/information_pricing_${SLURM_JOB_ID}/RESULTS_ONLY" \
    --flavor "$focal_flavor" --nodes 9 --delta 1
```

K Complete server data policy

Source: `Code/privacy/server_data_policy.py`.

SHA-256: `086b26b485e0943428f290282b8f032dafb52de3b62d8edab64d310c14b844b4`.

```
"""Fail closed before empirical I/O on a non-server machine.

No environment-variable override is provided. Results-only readers and theoretical
calculations do not require this guard. Research data remain under the known
server filesystem; a copied /scratch path on a Mac does not authorize execution.
"""
from functools import wraps
from pathlib import Path
import sys
import inspect

SERVER_ROOT = Path('/scratch/dtm63837/Kilts_Panel')


def require_server():
    if sys.platform != 'linux' or not SERVER_ROOT.is_dir():
        raise PermissionError(
            'Empirical data access is server-only. Run this workflow on the research '
            'server; copy back only estimates, predictions without household IDs, '
            'and aggregate results. Local input/output overrides are not permitted.')


def require_data_path(path):
    resolved = Path(path).expanduser().resolve()
    checkout = Path(__file__).resolve().parents[2]
    if not resolved.is_relative_to(SERVER_ROOT) or resolved.is_relative_to(checkout):
        raise PermissionError('Research inputs and outputs must stay on the server, outside LOV_CC_2YP.')


def require_legacy_server():
    require_server()
    require_data_path(Path.cwd())


def server_only(function):
    @wraps(function)
    def guarded(*args, **kwargs):
        require_server()
        bound = inspect.signature(function).bind_partial(*args, **kwargs)
        path_names = {'sample', 'data_dir', 'output', 'output_path', 'directory', 'work',
                      'household_input', 'weekly_output', 'input_path', 'path', 'retail_path',
                      'reference_path', 'products_path', 'flavors', 'products_extra',
                      'raw_dir', 'output_dir', 'panel_raw', 'retail_raw', 'retail_merged_input',
                      'recovery_output', 'destination'}
        for name, value in bound.arguments.items():
            if name in path_names and isinstance(value, (str, Path)):
                require_data_path(value)
            if name in {'args', 'settings'} and hasattr(value, '__dict__'):
                for key, path in vars(value).items():
                    if key in path_names and isinstance(path, (str, Path)):
                        require_data_path(path)
        return function(*args, **kwargs)
    return guarded
```

L Complete binary theoretical information exercise

Source: `Code/final_draft/writeup/1108_writeup/theory/gamma_uncertainty.py`.

SHA-256: `2613001787328347902 6ff844be08fef70565ec84f63d1efdd49bc6f9909ab0e`.

```python
"""Theoretical type uncertainty; no empirical records or inputs.
Run from any directory. Outputs go beside this script in ../assets/results/gamma_uncertainty.
Initial history is observed; gamma is known or evaluated using fixed type probabilities in each state.
"""
from pathlib import Path
import os
os.environ.setdefault('MPLCONFIGDIR', '/tmp/gamma-uncertainty-mpl')
import json
import argparse
import numpy as np
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[4] / 'final_simple/supply'))
from pricing_model import expit, golden_maximum, static_price
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

GAMMA=np.array([-1.25,0.,1.25]); BETA=2.; ALPHA=-1.; DELTA=1.0; COST=0.
OUT=Path(__file__).resolve().parents[1]/'assets/results/gamma_uncertainty'


def maximize(fun, wide=False):
    # Mixture objectives can have multiple stationary points: refine all grid modes.
    lo,hi=(-20.,40.) if wide else (-10.,20.)
    x=np.linspace(lo,hi,121 if wide else 81)
    y=np.array([fun(p) for p in x])
    candidates=[(y[0],x[0]),(y[-1],x[-1])]
    for k in range(1,len(x)-1):
        if y[k]>=y[k-1] and y[k]>=y[k+1]:
            price,value=golden_maximum(fun,x[k-1],x[k+1])
            candidates.append((value,price))
    value,p=max(candidates)
    assert lo+1e-4<p<hi-1e-4,(p,value)
    return float(p),float(value)


def shares(p,h,gamma=GAMMA):
    return expit(BETA+ALPHA*p+(1 if h=='0' else -1)*gamma)


def terminal(weights,h,wide=False):
    # Fast bracketed derivative for this calibration. Its negative derivative is
    # checked at the solution; the independent grid maximizer audits every case.
    fun=lambda p: float(np.dot(weights,shares(p,h))*(p-COST))
    derivative=lambda p: float(np.dot(weights,shares(p,h)+(p-COST)*ALPHA*shares(p,h)*(1-shares(p,h))))
    lo,hi=0.,20.
    for _ in range(100):
        mid=(lo+hi)/2
        if derivative(mid)>0: lo=mid
        else: hi=mid
    p=(lo+hi)/2
    return p,fun(p)


def known(h,prior):
    rows=[]
    for k,g in enumerate(GAMMA):
        p2={};values={}
        for a in ['j','0']:
            p2[a]=static_price(BETA+(1 if a=='0' else -1)*g,ALPHA,COST)
            values[a]=(p2[a]-COST)*float(shares(p2[a],a,g))
        effective=COST-DELTA*(values['j']-values['0'])
        p=static_price(BETA+(1 if h=='0' else -1)*g,ALPHA,effective);s=float(shares(p,h,g))
        value=s*(p-COST)+DELTA*(s*values['j']+(1-s)*values['0'])
```

```
        rows.append(dict(gamma=float(g),p1=p,p2j=p2['j'],p20=p2['0'],s1=s,profit=value))
    expected_p1=sum(prior[k]*r['p1'] for k,r in enumerate(rows))
    expected_p2=sum(prior[k]*(r['s1']*r['p2j']+(1-r['s1'])*r['p20']) for k,r in enumerate(rows))
    value=sum(prior[k]*r['profit'] for k,r in enumerate(rows))
    return dict(profit=value,expected_p1=expected_p1,expected_p2=expected_p2,types=rows)


def evaluate(p,h,prior,details=False):
    a=float(prior@shares(p,h))
    p2j,vj=terminal(prior,'j');p20,v0=terminal(prior,'0')
    value=a*(p-COST)+DELTA*(a*vj+(1-a)*v0)
    if not details:return value
    return dict(profit=value,expected_p1=p,expected_p2=a*p2j+(1-a)*p20,
                p2j=p2j,p20=p20,s1=a,belief_j=prior.tolist(),belief_0=prior.tolist())


def persistent_revenue(policy, h, prior):
    """Evaluate a fixed-belief policy with the SAME type in both periods.

    Policy selection retains fixed beliefs; evaluation accounts for the sorting
    of persistent types into first-period purchase outcomes.
    """
    s1 = shares(policy['expected_p1'], h)
    s2j = shares(policy['p2j'], 'j')
    s20 = shares(policy['p20'], '0')
    by_type = (s1*(policy['expected_p1']-COST)
               + DELTA*(s1*s2j*(policy['p2j']-COST)
                        + (1-s1)*s20*(policy['p20']-COST)))
    return float(np.asarray(prior) @ by_type)


def solve(prior,h,validate=True):
    prior=np.asarray(prior,dtype=float)
    assert np.isclose(prior.sum(),1) and (prior>=0).all()
    p,value=maximize(lambda p:evaluate(p,h,prior))
    unknown=evaluate(p,h,prior,True);full=known(h,prior)
    unknown['fixed_belief_objective'] = value
    unknown['persistent_type_revenue'] = persistent_revenue(unknown, h, prior)
    full['persistent_type_revenue'] = full['profit']
    assert full['profit']+1e-8 >= unknown['persistent_type_revenue']
    assert unknown['belief_j']==prior.tolist()==unknown['belief_0']
    if validate:
        pwide,vwide=maximize(lambda p:evaluate(p,h,prior),wide=True)
        assert abs(value-vwide)<1e-7
        # Independent direct optimization audits both terminal branches.
        s=shares(p,h)
        for state,w,price in [('j',prior,unknown['p2j']),('0',prior,unknown['p20'])]:
            pp,vv=maximize(lambda x:float(w@shares(x,state)*(x-COST)),wide=True)
            assert abs(vv-float(w@shares(price,state)*(price-COST)))<1e-8
        eps=1e-4
        assert abs((evaluate(p+eps,h,prior)-evaluate(p-eps,h,prior))/(2*eps))<1e-5
    return dict(prior=prior.tolist(),h0=h,known=full,unknown=unknown,value_of_information=full['profit']-unknown['
     persistent_type_revenue'])


def main():
    global OUT
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output', type=Path, default=OUT)
    OUT = parser.parse_args().output
    OUT.mkdir(parents=True,exist_ok=True)
    priors=[('Equal types',[1/3]*3),('Habit type more likely',[.5,.25,.25]),('Neutral type more likely',[.25,.5,.25])
     ,('Novelty type more likely',[.25,.25,.5])]
    results=[]
    for label,prior in priors:
        for h in ['j','0']:
            r=solve(prior,h);r['scenario']=label;results.append(r)
    # If type is certain, both information regimes must coincide.
    for k in range(3):
        for h in ['j','0']:
            prior=np.eye(3)[k];r=solve(prior,h)
```

```
            assert abs(r['value_of_information'])<1e-8
    (OUT/'solutions.json').write_text(json.dumps(results,indent=2))
    (OUT/'settings.json').write_text(json.dumps(dict(gamma=GAMMA.tolist(),beta=BETA,alpha=ALPHA,delta=DELTA,c=COST,
     priors=priors,revenue_evaluation='same persistent types under both policies; common profit field remains fixed-
     belief objective',checks='Wider first-price grids; independent terminal maximizers; first-price finite-
     difference FOCs; Fixed-belief invariance; full-information bound; degenerate priors'),indent=2))
    sys.path.insert(0, str(Path(__file__).resolve().parents[4]))
    from pricing_presentation import information_figures
    import pandas as pd
    table = pd.DataFrame([dict(scenario=r['scenario'], h0=r['h0'], case=case,
        expected_p1=r[case]['expected_p1'], expected_p2=r[case]['expected_p2'],
        persistent_type_revenue=r[case]['persistent_type_revenue'], value_of_information=r['value_of_information'])
        for r in results for case in ['known', 'unknown']])
    table.to_csv(OUT/'information_friction_aggregate.csv', index=False)
    type_rows = []
    for r in results:
        for label, row in zip(['low', 'medium', 'high'], r['known']['types']):
            type_rows.append(dict(scenario=r['scenario'], h0=r['h0'], type=label,
                gamma=row['gamma'], known_p1=row['p1'],
                known_expected_p2=row['s1']*row['p2j']+(1-row['s1'])*row['p20']))
    pd.DataFrame(type_rows).to_csv(OUT/'information_friction_by_group.csv', index=False)
    information_figures(table, OUT, DELTA)
    # Existing slide filenames remain valid; all price figures show p1 -> p2.
    import shutil
    for name in ['profits', 'first_prices', 'terminal_prices', 'information_value']:
        for extension in ['pdf', 'png']:
            shutil.copyfile(OUT/f'information_friction_{name}.{extension}', OUT/f'{name}.{extension}')
    lines=[r'\begin{table}[htbp]\centering\small',r'\caption{Pricing with known and uncertain switching types}',r'\
     label{tab:gamma-uncertainty}',r'\begin{tabular}{llrrr}\toprule',r'Initial state & Prior & Personalized pricing &
      Common pricing & Information value\\\midrule']
    for r in results:lines.append(f"${r['h0']}$ & {r['scenario']} & {r['known']['profit']:.4f} & {r['unknown']['
     persistent_type_revenue']:.4f} & {r['value_of_information']:.4f} \\\\")
    lines += [r'\bottomrule\end{tabular}',r'\end{table}'];(OUT/'results.tex').write_text('\n'.join(lines)+'\n')
    print('All checks passed; wrote four figures, aggregate theoretical solutions, settings, and result table.')

if __name__=='__main__':main()
```

M Theoretical numerical helper functions

Source: `Code/final_simple/supply/pricing_model.py`.

SHA-256: `f0ff34d74d7f20c76f83884536b95adb7da3ce69d6f92ba7f9622d9b5fa09f5c`.

M.1 `expit`

```python
def expit(x):
    x = np.asarray(x)
    small = np.exp(-np.abs(x))
    return np.where(x >= 0, 1/(1+small), small/(1+small))
```

M.2 `golden_maximum`

```python
def golden_maximum(function, lo, hi):
    ratio = (math.sqrt(5)-1)/2
    left, right = hi-ratio*(hi-lo), lo+ratio*(hi-lo)
    fl, fr = function(left), function(right)
    for _ in range(100):
        if hi-lo <= 1e-10*(1+abs(left)+abs(right)):
            break
        if fl > fr:
            hi, right, fr = right, left, fl
            left = hi-ratio*(hi-lo)
            fl = function(left)
        else:
            lo, left, fl = left, right, fr
            right = lo+ratio*(hi-lo)
            fr = function(right)
    return (left, fl) if fl > fr else (right, fr)
```

M.3 `static_price`

```python
def static_price(intercept, alpha, effective_cost=0.):
    """Signed alpha<0; monotone transformed FOC, including introductory subsidies."""
    def foc(p):
        return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
    lo, hi = effective_cost, effective_cost+max(2., 2/abs(alpha))
    while foc(hi) > 0:
        hi = lo+2*(hi-lo)
    for _ in range(100):
        mid = (lo+hi)/2
        if foc(mid) > 0:
            lo = mid
        else:
            hi = mid
    return (lo+hi)/2
```

N Complete binary regression and mathematical tests

Source: `Code/final_simple/checks/check_binary_information_pricing.py`.

SHA-256: `5d2d0426ad2b8054d565e13441d630ba27de1627c76e620f402a8a4b00843ecb`.

```
"""Binary mathematical checks only; no household records or empirical I/O."""
from pathlib import Path
import sys
import unittest
import numpy as np
from scipy.special import expit, logsumexp, wrightomega
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from demand.binary_information_pricing_model import solve_binary
from demand.gamma_calibration import known_prices, uncertainty


def setup(**changes):
    values = dict(prices=np.array([1., 1.4, 1.2]), available=np.ones(3, bool),
                  beta=np.array([1.2, 1.5, 1.]), stock=np.array([.2, .8, .1]),
                  alpha=1.1, habit=.2, gamma=np.array([-1.25, 0., 1.25]),
                  weights=np.ones(3)/3, initial_inside=True)
    values.update(changes)
    return values


class BinaryPricingChecks(unittest.TestCase):
    def test_exact_supply_model_limit(self):
        for state in ['j', '0']:
            env = setup(prices=np.array([1.]), available=np.ones(1, bool),
                        beta=np.array([2.]), stock=np.zeros(1), focal=0,
                        alpha=1., habit=0., initial_inside=(state == 'j'))
            actual = solve_binary(**env)
            for k, g in enumerate(env['gamma']):
                expected = known_prices(g, state, -1., 2., 1.)
                np.testing.assert_allclose([actual['full']['p1'][k], actual['full']['expected_p2'][k],
                                            actual['full']['revenue'][k]],
                                           [expected['expected_p1'], expected['expected_p2'], expected['profit']],
     atol=2e-7)
            common = uncertainty(env['gamma'], env['weights'], state, -1., 2., 1.)
            np.testing.assert_allclose(actual['limited']['p1'], common['expected_p1'], atol=2e-7)
            np.testing.assert_allclose(actual['p2_limited'], [common['p2j'], common['p20']], atol=2e-7)
            self.assertAlmostEqual(actual['limited_objective'], common['profit'], places=7)

    def test_two_choices_two_successors(self):
        for size in [3, 5]:
            result = solve_binary(**setup(prices=np.ones(size), available=np.ones(size, bool),
                                          beta=np.ones(size), stock=np.zeros(size)))
            np.testing.assert_array_equal(result['states'], [[1, 0], [0, 1]])
            self.assertEqual(result['full']['state_probability'].shape, (3, 2))
            np.testing.assert_allclose(result['full']['state_probability'].sum(axis=1), 1.)
            self.assertEqual(result['p2_limited'].shape, (2,))

    def test_outside_mapping_and_habit(self):
        env = setup()
        result = solve_binary(**env)
        outside = logsumexp(np.r_[env['beta'][[0, 2]]-env['alpha']*env['prices'][[0, 2]]
                                  +env['habit']*env['stock'][[0, 2]], 0.])
        self.assertAlmostEqual(result['outside_index'], outside)
        # h=j gives -gamma, h=0 gives +gamma; own habit rises only after plain.
        expected = result['binary_intercept']+np.column_stack([
            -env['gamma']+env['habit']*(env['stock'][1]+1),
            env['gamma']+env['habit']*env['stock'][1]])
        np.testing.assert_allclose(result['terminal_utility_intercept'], expected)

    def test_type_certain_regimes_agree(self):
        result = solve_binary(**setup(gamma=np.array([-.8]), weights=np.ones(1)))
        self.assertAlmostEqual(result['gain'], 0., places=9)
        np.testing.assert_allclose(result['full']['p1'], result['limited']['p1'], atol=1e-8)
        np.testing.assert_allclose(result['p2_full'][0], result['p2_limited'], atol=1e-8)
```

```
64
65      def test_dense_independent_first_price_search_and_evaluation(self):
66          for initial in [False, True]:
67              env = setup(initial_inside=initial, habit=.7)
68              result = solve_binary(**env)
69              grid = np.linspace(-5., 10., 20001)
70              for k in range(3):
71                  q = expit(result['first_utility_intercept'][k]-env['alpha']*grid)
72                  values = grid*q+q*result['terminal_revenue_full'][k, 0]+(1-q)*result['terminal_revenue_full'][k, 1]
73                  self.assertGreaterEqual(result['full']['revenue'][k]+1e-9, values.max())
74                  common_p1 = result['limited']['p1'][k]
75                  s1 = expit(result['first_utility_intercept'][k]-env['alpha']*common_p1)
76                  revenue = common_p1*s1+s1*result['terminal_revenue_limited'][k, 0]+(1-s1)*result['
       terminal_revenue_limited'][k, 1]
77                  self.assertAlmostEqual(revenue, result['limited']['revenue'][k], places=10)
78              qbar = env['weights'] @ expit(result['first_utility_intercept'][:, None]-env['alpha']*grid)
79              perceived = env['weights'] @ result['terminal_revenue_limited']
80              subjective = grid*qbar+qbar*perceived[0]+(1-qbar)*perceived[1]
81              self.assertGreaterEqual(result['limited_objective']+1e-9, subjective.max())
82              self.assertGreaterEqual(result['gain'], -1e-10)
83              np.testing.assert_array_equal(result['limited_beliefs'], np.ones((3, 2))/3)
84
85      def test_no_continuation_and_negative_habit(self):
86          for phi in [-.4, .4]:
87              result = solve_binary(**setup(delta=0., habit=phi))
88              expected = (1+wrightomega(result['first_utility_intercept']-1))/1.1
89              np.testing.assert_allclose(result['full']['p1'], expected, atol=1e-9)
90              self.assertGreaterEqual(result['gain'], -1e-10)
91
92      def test_focal_permutation_and_unavailable_rival(self):
93          env = setup(); reference = solve_binary(**env)
94          order = [0, 2, 1]
95          for key in ['prices', 'available', 'beta', 'stock']:
96              env[key] = env[key][order]
97          permuted = solve_binary(**env, focal=2)
98          np.testing.assert_allclose(reference['full']['revenue'], permuted['full']['revenue'], atol=1e-9)
99          one = solve_binary(**setup(available=np.array([False, True, False])))
100         self.assertAlmostEqual(one['outside_index'], 0.)
101
102 if __name__ == '__main__':
103     unittest.main()
```

O Previous full-basket solver: the source of the earlier results

This is preserved for comparison, not called by the corrected binary runner. Its full `solve_environment` enumeration explains why the earlier output was not the requested binary exercise.

```python
"""Pure two-period price model: only novelty is hidden, habit and price taste known.

Inputs are mathematical utility/history arrays. This module performs no empirical
I/O and creates no household records. The server runner supplies fitted contexts.
"""
from math import lgamma
import numpy as np
from scipy.special import expit, logsumexp, wrightomega

try:
    from .history import update_history
except ImportError:
    from history import update_history


def baskets(capacity, inside):
    def generate(total, slots):
        if slots == 1:
            yield (total,)
        else:
            for n in range(total+1):
                for rest in generate(total-n, slots-1):
                    yield (n,)+rest
    return np.array(list(generate(capacity, inside+1)), dtype=int)


def _scalar_maximum(value, derivative, bound, points):
    """Screen stationary points on a dimensionless grid and refine every maximum."""
    for _ in range(8):
        count = max(points, int(8*bound)+1)
        if count > 32769:
            raise ValueError('Price search too wide; inspect the fitted utility scale.')
        grid = np.linspace(-bound, bound, count)
        slopes = derivative(grid)
        if slopes[0] > 0 and slopes[-1] < 0:
            break
        bound *= 2
    else:
        raise ArithmeticError('Could not bracket first-period price optimum.')
    left = grid[:-1][(slopes[:-1] >= 0) & (slopes[1:] <= 0)]
    right = grid[1:][(slopes[:-1] >= 0) & (slopes[1:] <= 0)]
    for _ in range(48):
        mid = (left+right)/2
        positive = derivative(mid) > 0
        left, right = np.where(positive, mid, left), np.where(positive, right, mid)
    candidates = np.r_[grid, (left+right)/2]
    values = value(candidates)
    best = int(np.argmax(values))
    # Near an optimum, objective differences are second order. Rounding can make
    # a grid point look one ulp better while its first-order residual is larger.
    # Prefer a refined stationary point when values differ only by roundoff.
    if len(left):
        root_best = len(grid)+int(np.argmax(values[len(grid):]))
        tolerance = 64*np.finfo(float).eps*max(abs(values[best]), abs(values[root_best]),
                                               np.finfo(float).tiny)
        if values[root_best] >= values[best]-tolerance:
            best = root_best
    if candidates[best] in [grid[0], grid[-1]]:
        raise ArithmeticError('First-period optimum touches search boundary.')
    return float(candidates[best]), float(values[best])


def _terminal_common(intercepts, weights, alpha, capacity, points):
    """Fixed-belief terminal optimum for every basket, shared across novelty types.

```

```
    Optimize in t=alpha*p. Every mixture optimum lies between the component
    optima. Grid-screen all local maxima and bisect all positive-to-negative FOCs.
    """
    component = 1+wrightomega(intercepts-1)
    lo, hi = component.min(axis=0), component.max(axis=0)
    count = max(points, int(8*np.max(hi-lo))+1)
    if count > 32769:
        raise ValueError('Terminal mixture too wide for the price search.')
    grid = lo[None, :]+np.linspace(0, 1, count)[:, None]*(hi-lo)[None, :]
    def objective(t, states):
        q = expit(intercepts[:, states]-t[None, :])
        return capacity/alpha*t*(weights[:, None]*q).sum(axis=0)
    def slope(t, states):
        q = expit(intercepts[:, states]-t[None, :])
        return (weights[:, None]*q*(1-t[None, :]*(1-q))).sum(axis=0)
    q = expit(intercepts[:, None, :]-grid[None, :, :])
    d = (weights[:, None, None]*q*(1-grid[None, :, :]*(1-q))).sum(axis=0)
    values = capacity/alpha*grid*(weights[:, None, None]*q).sum(axis=0)
    selected = values.argmax(axis=0)
    states_all = np.arange(len(lo))
    price_t = grid[selected, states_all].copy()
    best = values[selected, states_all].copy()
    rows, states = np.where((d[:-1] >= 0) & (d[1:] <= 0))
    left, right = grid[rows, states], grid[rows+1, states]
    for _ in range(48):
        mid = (left+right)/2
        positive = slope(mid, states) > 0
        left, right = np.where(positive, mid, left), np.where(positive, right, mid)
    roots = (left+right)/2
    root_values = objective(roots, states)
    for state, root, value in zip(states, roots, root_values):
        # Prefer the refined root to a revenue-equivalent grid point. A strict
        # comparison can retain the grid point solely because of rounding.
        tolerance = 64*np.finfo(float).eps*max(abs(best[state]), abs(value),
                                               np.finfo(float).tiny)
        if value >= best[state]-tolerance:
            best[state], price_t[state] = value, root
    np.testing.assert_allclose(slope(price_t, states_all), 0., atol=2e-9)
    return price_t/alpha, best


def solve_environment(prices, available, beta, novelty, stock, alpha, habit,
                      gamma, weights, capacity=7, focal=1, delta=1.,
                      stock_measure='cumulative', stock_retention=1., occasions=1,
                      controls=None, search_points=65):
    """Solve one known-price/habit/history environment across persistent gamma types.

    Known regimes share exactly the same utility and history transition. Complete
    information knows gamma; limited information uses fixed weights at each state.
    The latter first-price objective is the theory's fixed-belief objective.
    Both policies are evaluated using the SAME persistent type in both periods.
    Common means common across gamma, conditional on known alpha, habit, history,
    rival prices and menus; it does not mean one dollar price across households.
    All types are priced; there is no middle-group price restriction. First prices
    can be negative; terminal prices are nonnegative at zero marginal cost.
    """
    arrays = [np.asarray(x, float) for x in [prices, beta, novelty, stock, gamma, weights]]
    prices, beta, novelty, stock, gamma, weights = arrays
    available = np.asarray(available, bool)
    j = len(prices)
    if (any(x.shape != (j,) for x in [beta, novelty, stock, available])
            or gamma.ndim != 1 or weights.shape != gamma.shape or len(gamma) < 1
            or not all(np.isfinite(x).all() for x in arrays)
            or not np.isfinite([alpha, habit, delta]).all() or alpha <= 0
            or (weights < 0).any() or not np.isclose(weights.sum(), 1.)
            or (prices < 0).any() or not 0 <= delta <= 1
            or int(capacity) != capacity or not 1 <= capacity <= 12
            or not 0 <= focal < j or not available[focal] or search_points < 17):
        raise ValueError('Invalid pricing dimensions, probabilities, menu or parameters.')
    capacity = int(capacity)
    controls = np.zeros(j) if controls is None else np.asarray(controls, float)
```

```
    if controls.shape != (j,) or not np.isfinite(controls).all():
        raise ValueError('Invalid fixed utility controls.')
    # Enumerate only available alternatives, retaining full shares for history.
    active = np.flatnonzero(available)
    local_focal = int(np.flatnonzero(active == focal)[0])
    choices = np.r_[active, j]  # outside/nonpurchase has fixed utility zero
    states_local = baskets(capacity, len(active))
    states = np.zeros((len(states_local), j+1), dtype=int)
    states[:, choices] = states_local
    inside_total = states[:, :j].sum(axis=1, keepdims=True)
    shares = np.divide(states[:, :j], inside_total,
                       out=np.zeros_like(states[:, :j], dtype=float), where=inside_total > 0)
    novelty2 = shares.sum(axis=1, keepdims=True)-shares
    stock2 = update_history(stock, shares, occasions, measure=stock_measure, retention=stock_retention)
    first = beta+gamma[:, None]*novelty+habit*stock+controls-alpha*prices
    terminal = (beta[None, None, :]+gamma[:, None, None]*novelty2[None, :, :]
                +habit*stock2[None, :, :]+controls[None, None, :]-alpha*prices)
    first[:, focal] += alpha*prices[focal]
    terminal[:, :, focal] += alpha*prices[focal]
    nonfocal = active[active != focal]
    competitor1 = np.column_stack([first[:, nonfocal], np.zeros(len(gamma))])
    intercept1 = first[:, focal]-logsumexp(competitor1, axis=1)
    competitor2 = np.concatenate([terminal[:, :, nonfocal],
                                  np.zeros((*terminal.shape[:2], 1))], axis=-1)
    intercept2 = terminal[:, :, focal]-logsumexp(competitor2, axis=-1)
    p2_full = (1+wrightomega(intercept2-1))/alpha
    q2_full = expit(intercept2-alpha*p2_full)
    r2_full = capacity*p2_full*q2_full
    p2_limited, perceived2 = _terminal_common(intercept2, weights, alpha, capacity, search_points)
    r2_limited = capacity*p2_limited[None, :]*expit(intercept2-alpha*p2_limited)
    # Given focal count n, relative probabilities of all other alternatives do
    # not depend on the focal price. Aggregate continuation by n to optimize p1.
    n = states[:, focal]
    others = np.r_[nonfocal, j]
    log_cond = competitor1-logsumexp(competitor1, axis=1, keepdims=True)
    remaining_counts = states[:, others]
    coefficient = np.array([lgamma(capacity-int(k)+1)-sum(lgamma(int(c)+1) for c in row)
                            for k, row in zip(n, remaining_counts)])
    conditional = np.exp(coefficient[None, :]+log_cond @ remaining_counts.T)
    c_full = np.zeros((len(gamma), capacity+1))
    c_perceived = np.zeros_like(c_full)
    for k in range(capacity+1):
        mask = n == k
        np.testing.assert_allclose(conditional[:, mask].sum(axis=1), 1., atol=1e-12)
        c_full[:, k] = (conditional[:, mask]*r2_full[:, mask]).sum(axis=1)
        c_perceived[:, k] = conditional[:, mask] @ perceived2[mask]
    counts = np.arange(capacity+1)
    binomial_coef = np.array([lgamma(capacity+1)-lgamma(int(k)+1)-lgamma(capacity-int(k)+1)
                              for k in counts])
    def binomial(t, intercept):
        t = np.atleast_1d(t)
        logq = -np.logaddexp(0., t-intercept)
        logone = -np.logaddexp(0., intercept-t)
        mass = np.exp(binomial_coef+logq[:, None]*counts+logone[:, None]*(capacity-counts))
        return expit(intercept-t), mass
    def functions(intercepts, continuation, w):
        def value(t):
            result = np.zeros(np.atleast_1d(t).shape)
            for z, c, weight in zip(intercepts, continuation, w):
                q, mass = binomial(t, z)
                result += weight*(capacity*q*np.atleast_1d(t)/alpha+delta*(mass @ c))
            return result
        def slope(t):
            result = np.zeros(np.atleast_1d(t).shape)
            for z, c, weight in zip(intercepts, continuation, w):
                q, mass = binomial(t, z)
                continuation_slope = (mass*(capacity*q[:, None]-counts)) @ c
                result += weight*(capacity/alpha*q*(1-np.atleast_1d(t)*(1-q))
                                  +delta*continuation_slope)
            return result
        return value, slope
```

```
    bound = max(10., float(np.max(np.abs(intercept1)))+5.,
                float(np.max(alpha*r2_full))/capacity+5.)
    p1_full = np.empty(len(gamma))
    for k in range(len(gamma)):
        val, derivative = functions(intercept1[k:k+1], c_full[k:k+1], np.ones(1))
        p1_full[k] = _scalar_maximum(val, derivative, bound, search_points)[0]/alpha
    val, derivative = functions(intercept1, c_perceived, weights)
    limited_t, limited_objective = _scalar_maximum(val, derivative, bound, search_points)
    p1_limited = np.full(len(gamma), limited_t/alpha)
    def evaluate(p1, p2, r2):
        mass = np.empty((len(gamma), len(states)))
        r1, revenue2, expected2 = [], [], []
        for k in range(len(gamma)):
            q, bin_mass = binomial(np.array([alpha*p1[k]]), intercept1[k])
            mass[k] = conditional[k]*bin_mass[0, n]
            r1.append(capacity*q[0]*p1[k])
            revenue2.append(mass[k] @ r2[k])
            expected2.append(mass[k] @ p2[k])
        np.testing.assert_allclose(mass.sum(axis=1), 1., atol=1e-10)
        return dict(p1=p1, expected_p2=np.array(expected2), revenue1=np.array(r1),
                    revenue2=np.array(revenue2), revenue=np.array(r1)+delta*np.array(revenue2),
                    state_probability=mass)
    full = evaluate(p1_full, p2_full, r2_full)
    limited = evaluate(p1_limited, np.broadcast_to(p2_limited, p2_full.shape), r2_limited)
    # Every limited-information price path is feasible for each informed type.
    if (full['revenue']+1e-7 < limited['revenue']).any():
        raise AssertionError('Known-type optimization fails to dominate common policy.')
    return dict(full=full, limited=limited, gamma=gamma, weights=weights, states=states,
                p2_full=p2_full, p2_limited=p2_limited,
                terminal_revenue_full=r2_full, terminal_revenue_limited=r2_limited,
                limited_beliefs=np.broadcast_to(weights[:, None], (len(gamma), len(states))).copy(),
                limited_objective=limited_objective,
                full_revenue=float(weights @ full['revenue']),
                limited_revenue=float(weights @ limited['revenue']),
                gain=float(weights @ (full['revenue']-limited['revenue'])))
```